

Design and Implementation of a Vision-Guided Robotic System for Automated Vegetable Sorting

Name: Fathhur Rahaman Sams

ID: 2022-2-60-041

Name: Sanjana Kazi Supti

ID: 2022-2-60-036

Name: Md Junaeid Ali

ID:2022-2-60-127

Name: Mahfuj Alam Imon

ID:2022-2-60-131

**A Capstone project report submitted in partial fulfillment of the requirements
for the degree of Bachelor of Science in Computer Science and Engineering**



**Department of Computer Science and Engineering
East West University
Dhaka-1212, Bangladesh**

April, 2026

Declaration

We, **Fathhur Rahaman Sams, Sanjana Kazi Supti, Md Junaeid Ali, Mahfuj Alam Imon**, herewith, announce that the work presented in this capstone project report is the original outcome of our investigation, conducted under the valued guidance of our supervisor, **Rashedul Amin Tuhin**, Designation, Department of Computer Science and engineering, East West University. Additionally We are confirming that no portion of this project has been or is currently being, submitted to any other institution for the award of a degree or diploma except for publication.

Countersigned

Signature

.....
Rashedul Amin Tuhin
Supervisor

.....
Md. Fathhur Rahaman Sams
2022-2-60-041
Signature

.....
Sanjana Kazi Supti
2022-2-60-036
Signature

.....
Md Junaeid Ali
2022-2-60-127
Signature

.....
Mahfuj Alam Imon
2022-2-60-131

Letter of Acceptance

The capstone project report entitled "Design and Implementation of a Vision-Guided Robotic System for Automated Vegetable Sorting" is submitted by **Fathhur Rahaman Sams, Sanjana Kazi Supti, Md Junaeid Ali, Mahfuj Alam Imon** to the Department of Computer Science and Engineering, East West University, Dhaka, Bangladesh is accepted for the partial requirement for the completion of the degree of Bachelor of Science in Computer Science and Engineering on (17/02/2025).

Board of Examiners

1. _____
Rashedul Amin Tuhin
Senior Lecturer
Department of Computer Science and Engineering
East West University

2. _____
Dr. Maheen Islam
Chairperson , Associate Professor
Department of Computer Science and Engineering
East West University

Abstract

Vegetable sorting, which requires accuracy and timeliness, has always been a challenge in post-harvest processing, where human evaluation through visual inspection has been prone to inconsistencies, higher labor costs, and unnecessary vegetable waste. This paper describes the conceptualization and complete development of a vision-guided robotic system for automating vegetable sorting using self-supervised representation learning in conjunction with real-time object detection and robotic manipulation. A well-balanced dataset of 4,640 images was created across eight vegetable types: cucumber, tomato, bitter melon, pointed melon, potato, onion, eggplant, and lemon, each further divided into fresh and damaged subcategories according to different spatial configurations and multi-label settings. To set up the baselines for detection, three supervised models, namely YOLOv10s, YOLOv11s, and YOLOv12s, were trained in the same environment. Among them, YOLOv12s performed the best with 98.8% precision, 97.9% recall, 99.2% mAP@0.5, and 85.7% mAP@0.5:0.95. To make the models less reliant on large-scale annotated data and enhance the robustness of the features, self-supervised pretraining techniques such as DINOv2-style learning and BYOL were employed before fine-tuning the detectors. Among the self-supervised pretraining techniques, the DINOv2-style pretraining of YOLOv12s showed the highest overall performance, reaching 99.1% precision, 98.5% recall, 99.6% mAP@0.5, and 88.3% mAP@0.5:0.95, demonstrating improved localization stability under varying lighting and occlusion conditions. The perception module was coupled with a 6-DOF Hiwonder xArm 1S robotic arm (320mm reach, 300g payload capacity, 1mm positioning accuracy) with six intelligent servo motors and a parallel gripper to enable autonomous pick-and-place sorting based on the class labels identified. The robotic platform carries out a set of grasp, lift, and place operations in a predefined drop location for each class, running in a closed-loop perception-action cycle. The entire system, from real-time SSL-based detection to AI explainability and robotic control, is implemented using an interactive Streamlit dashboard. The experimental results verify that the detection robustness is greatly improved with the help of self-supervised pretraining, which also enables the real-time robotic sorting task. It has been demonstrated that the proposed system can leverage the benefits of the accurate and lightweight robotic manipulation, along with the annotation-efficient self-supervised learning, to deliver a cost-effective solution for the sorting of vegetable freshness in the agricultural processing industry.

Acknowledgments

Like anyone reaching an important milestone, we too have come this far thanks to the support, guidance, and interactions with many individuals. Expressing gratitude in writing can be challenging, as words often fail to capture the depth of appreciation even in one's native language. So rather than searching for perfect phrases, we've chosen to simply acknowledge those whose contributions were vital to the completion of this thesis. This research was conducted within the Department of Computer Science and Engineering at East West University, Bangladesh.

First and foremost, we are sincerely thankful to the Almighty for His continuous blessings. We would also like to extend our heartfelt appreciation to our supervisor, Rashedul Amin Tuhin, who gave us this opportunity, initiated us into the field of "Vision Guided Robotic Vegetable Sorting System", and this project would not have been possible without the consistent support and guidance of our supervisor. His thoughtful advice, encouragement, and ability to push us toward independent thinking have left a lasting impact on our own future paths.

We are also thankful to our group members for their remarkable teamwork during the performance evaluation phase. Their continuous support, shared knowledge, and practical advice were instrumental in navigating complex technical issues. We extend our heartfelt appreciation to our parents, whose endless encouragement, prayers, and belief in us have been a constant source of strength.

Finally, we recognize the many individuals who, in ways both direct and subtle, have offered their friendship and support throughout our academic life. Though not all are named here, we hold them in high regard and hope to properly acknowledge their impact on time.

Md. Fathhur Rahaman Sams
April 2026

Sanjana Kazi Supti
April 2026

Md Junaeid Ali
April 2026

Mahfuj Alam Imon
April 2026

Table of Contents

Declaration.....	1
Letter of Acceptance.....	2
Abstract.....	3
Acknowledgment.....	4
Table of Contents.....	5
List of Figures.....	7
List of Tables.....	8
List of Algorithms.....	9
List of Acronyms	10
Chapter 1 Introduction	
1.1. Background	1
1.2. Problem Statement an Analysis	2
1.3. Project Objectives	2
1.4. Project Contributions	3
1.5. Project Outlines	4
 Chapter 2 Related Works	
2.1 Survey of the State-of-the-art	6
2.2 Summary	9
 Chapter 3 Materials and Method	
3.1 Materials	11
3.1.1 Dataset Collection SetUp	12
3.1.2 Dataset Collection Strategy	12
3.1.3 Dataset Description	14
3.1.4 Dataset Annotation and Preprocessing	17
3.1.5 Experimental Setup	19
3.1.5.1. Hardware Setup	19
3.1.5.2. Training Configuration	21
3.1.6 Evaluation Matrices	24
3.2 Method	26
3.2.1 Baseline Supervised Detection Model	26
3.2.2 Self Supervised Learning (SSL) Architectures	27
3.2.3 Explainable AI (XAI) Framework	32
3.2.4 Vision-Guided Robotic Automation Pipeline	33

3.2.4.1. System Architecture Overview	33
3.2.4.2. Workspace Calibration and Camera Coordination	34
3.2.4.3. Object Detection Framework	36
3.2.4.4. Tracking	37
3.2.4.5. Motion Planning and Servo Control	38
3.2.4.6. Autonomous Execution Strategy	40
3.2.5 Realtime Web Application Dashboard	42
3.3 Summary	43
 Chapter 4 Results and Discussion	
4.1 Performance Evaluation	44
4.1.1. Evaluation of Supervised Model Performance	44
4.1.2. Evaluation of SSL Architecture Performance	45
4.2 Loss Curve and Clustering Evaluation	47
4.2.1. Loss Curve Performance of Supervised Models	47
4.2.2. Loss and mAP Curve Performance of SSL Architectures	48
4.2.3. Clustering Analysis of SSL Architecture	49
4.2.4. Confusion Matrix Analysis	51
4.3 Detection Visualization of SSL Architectures	52
4.4 Explainable AI Techniques	53
4.5 Robotic arm Performance	54
4.6 Real-Time Web Application Dashboard	59
4.7 Discussion	60
4.7.1. Summary of Findings	60
4.7.2. Comparison with Current Literature	62
4.8 Summary	65
 Chapter 5 Conclusion	
5.1 Overall Contributions	67
5.2 Limitations and Future Works	67
 Bibliography	69
 Appendix	73

List of Figures

1. Workflow Diagram
2. Dataset Workflow
3. Classwise distribution Flow
4. Dataset Distribution for Object Detection
5. Visualized Images
6. YOLO Formatted Annotated Images
7. DINOv2 Style Architecture
8. BYOL Architecture
9. MAE Architecture
10. System Architecture Overview
11. System Workspace
12. State Machine Diagram
13. Robotic Pick and Place Arm with Vision
14. Robotic Functionality Workflow
15. Loss, Learning and mAP Curve of Baseline YOLO Pretrains
16. Loss and mAP Curve of SSL Architectures
17. Clustering of DINOv2 Style Architecture using YOLOv12s
18. Clustering of BYOL Architecture using YOLOv12s model
19. KNN Confusion Matrix after BYOL Feature Extraction
20. Vegetable Detection Visualized by SSL Architecture
21. EigenCAM Visualization and HeatMap
22. Streamlit Dashboard
23. Experimental Platform
24. Sequential Operational Workflow

List of Tables

1. Specifications of Mobile Devices Used for Image Acquisition
2. Data Collection Strategy Distribution
3. Dataset distribution for Single Class Classification images
4. Distribution of Dataset Annotation
5. Dataset Distribution for Object Detection
6. Specifications of the Hiwonder xArm 1S Robotic Arm
7. Comparative Hyperparameter Settings for Baseline Models
8. Comparative Hyperparameter Configuration Across SSL Architectures
9. Supervised Model Performance
10. SSL Architecture Performance With Best Baseline Backbone
11. Class-wise pick-and-place performance over a 5-minute evaluation period
12. Core system performance metrics computed from runtime execution
13. Detection and tracking performance metrics
14. Trajectory-based validation metrics for pick success verification
15. Task-based energy consumption metrics for pick-and-place operations
16. Analytical Comparison of Supervised and Self-Supervised Learning Outcomes Across Key Performance Dimensions
17. Comparison of Proposed Framework with Related Literature on Vegetable Detection, Self-Supervised Learning, and Agricultural Robotic

List of Algorithms

1. YOLOv10s
2. YOLOv11s
3. YOLOv12s
4. DINOv2-style
5. BYOL
6. MAE

List of Acronyms

YOLO	You Only Look Once
DINOv2-style	Distillation of knowledge with NO labels v2
BYOL	Bootstrap Your Own Latent
XAI	Explainable artificial intelligence
MAE	Masked Autoencoder

Chapter 1

Introduction

1.1 Background

In the world of farming and food today, sorting vegetables is still a hard job that needs a lot of people and is often done wrongly. People looking at the vegetables to decide if they are good or not is not a way to do this. It is not consistent and it costs a lot of money. It is not fast or accurate enough for the way food is sold and moved around nowadays. Vegetables are things that can go bad quickly so they need to be sorted by how fresh they are. This needs to be done quickly and correctly. This is important so that the people who buy the food are safe and so that a lot of money is not lost because the food goes bad. Now computers and robots are being used together to make a way to sort vegetables. These machines can look at the vegetables. See if they are bad or not and they can do this very quickly and correctly. This can help a lot because it can make the work of sorting vegetables more accurate and it can also help the environment. The world is having people and the weather is changing so the farming industry needs to find new ways to do things. New ways of using computers to look at things and robots to pick things up are already being used on farms to do jobs. These new ways are very good, at finding problems and picking up things. They are helping to make farming more automatic. Hyperspectral image analysis combined with deep learning has emerged as a highly reliable approach for rapid food quality measurement. Concurrently, improved YOLO-based object detection architectures have been demonstrated for robust fruit and vegetable detection in complex real-world environments. Multi-sensing fusion methods have further enhanced the estimation of ripeness for produce such as tomatoes. More recently, self-supervised and semi-supervised learning paradigms have gained traction as annotation-efficient alternatives, enabling models to learn expressive feature representations from unlabeled data and thereby reducing dependence on large manually annotated datasets. In the field of robotic manipulation, hybrid soft-rigid arm designs and optimized trajectory planning methods have shown considerable efficacy in agricultural pick-and-place tasks. Transformer-based architectures have additionally demonstrated effective maturity recognition and multi-source spectral fusion for fruit quality prediction, while edge-computing frameworks have enabled real-time detection for resource-constrained agricultural automation scenarios.

Despite these advances, a number of critical gaps persist in the literature that hinder the deployment of fully autonomous and scalable vegetable sorting systems. Existing supervised learning approaches are heavily reliant on large-scale annotated datasets, the preparation of which is time-consuming and burdensome, particularly across diverse vegetable varieties exhibiting varying freshness levels. The majority of prior studies address isolated components of the detection or manipulation pipeline without achieving end-to-end integration, resulting in degraded real-time performance under uncontrolled conditions such as active processing lines. Furthermore, the application of

self-supervised learning to multi-class freshness classification scenarios encompassing both fresh and spoiled samples remains underexplored, and the integration of low-cost robotic arms with in-situ cameras for precise and autonomous pick-and-place operations has received insufficient attention.

1.2 Problem Statement and Analysis

Sorting vegetables by how fresh they're is a big problem that involves computers, machines that can learn and robots that can pick things up. The systems we have now are not very good at doing this job in the world where food is actually being processed. There are reasons for this. First the computers need a lot of help to figure out what is fresh and what is not. They need people to label a lot of pictures of vegetables so they can learn.. Labeling all these pictures is a lot of work and it is hard to get enough pictures that show all the different things that can happen to vegetables. Second the computers have trouble when the pictures are not perfect. If the vegetables are blocked from view or the light is bad or there is stuff in the background the computers get confused. This happens a lot on the lines where vegetables are being sorted. Third we do not have a way to connect the computers that look at the pictures to the robots that pick up the vegetables. Usually people work on these two problems separately. Fourth we need to be able to understand how the computers are making their decisions about what's fresh and what is not. Now it is hard to know why the computers are doing what they are doing so people do not trust them very much. Fifth it is hard to tell the difference between vegetables that're fresh and ones that are spoiled when they look very similar. This is especially true for vegetables that're only a little bit spoiled and it is hard to see the difference. Vegetable freshness is a deal and we need to be able to sort vegetables by how fresh they are. Vegetable freshness sorting is important for freshness.

1.3 Project Objectives

This project is guided by three principal objectives, each targeting a distinct dimension of the automated vegetable sorting problem.

Objective 1: Development of a Robust Self-Supervised Learning Framework. To develop a robust self-supervised learning technique tailored for multi-class vegetable freshness categorization, with the aim of significantly enhancing detection accuracy utilizing extremely low numbers of labeled samples by leveraging unsupervised pretraining techniques applied to diverse collections of fresh and damaged vegetables. This encompasses the investigation of DINOv2-style self-distillation and BYOL-based contrastive learning as pretraining strategies, with subsequent fine-tuning of YOLO-family detectors on annotated subsets.

Objective 2: Architectural Optimization for Real-Time Robotic Sorting. To create and implement architectural modifications to AI detection models including

optimizations for real-time inference and compatibility with the Hiwonder xArm 1S robotic arm in order to achieve more than 95% sorting accuracy and enable efficient, safe pick-and-place operations under varied and dynamic imaging conditions. This objective encompasses the comparative evaluation of YOLOv10s, YOLOv11s, and YOLOv12s architectures under uniform training conditions, followed by integration of the highest-performing model into the robotic control pipeline.

Objective 3: Environmental Robustness and System Reliability. To identify how changes in key environmental conditions such as illumination variation, object occlusion, and conveyor belt speed influence overall system performance, and to recommend and implement pragmatic mitigation measures to increase reliability and scalability in real-world agricultural sorting deployments. This objective further encompasses the integration of explainable AI techniques, including Grad-CAM and Eigen-CAM, to ensure that model decisions are interpretable, transparent, and verifiable against ground-truth visual cues.

1.4 Project Contributions

This project makes several key contributions to agricultural computer vision, self-supervised learning, and vision-guided robotics. Custom Multi-Class Vegetable Freshness Dataset. We curated a balanced dataset of 4,640 images featuring eight non-seasonal vegetable types: cucumber, tomato, bitter melon, pointed gourd, potato, onion, eggplant, and lemon. Each vegetable type is divided into fresh and damaged subcategories. We captured images on five different mobile devices under controlled but varied conditions. Annotations were created using the Roboflow platform in YOLO format for both single-label and multi-label setups. Systematic Evaluation of YOLO-Family Supervised Baselines. We trained YOLOv10s, YOLOv11s, and YOLOv12s under the same conditions using the freshness dataset. YOLOv12s, which included area-attention modules and FlashAttention, achieved the best performance, reaching 98.8% precision, 97.9% recall, 99.2% mAP@0.5, and 85.7% mAP@0.5:0.95. This establishes a solid detection baseline. Integration of Self-Supervised Pretraining for Detection Improvement. We used DINOv2-style self-distillation, BYOL, and Masked Autoencoder (MAE) pretraining methods to initialize the YOLOv12s backbone before supervised fine-tuning. The DINOv2-style pretrained model offered the best overall performance with 99.1% precision, 98.5% recall, 99.6% mAP@0.5, and 88.3% mAP@0.5:0.95. It showed better localization stability under changing lighting and occlusion. Feature clustering analyses confirmed that DINOv2-pretrained embeddings produced more compact and distinct class representations compared to both BYOL and the supervised baseline. End-to-End Vision-Guided Robotic Sorting Pipeline. The perception module worked with a 6-DOF Hiwonder xArm 1S robotic arm, which operates in a 28 cm × 14 cm workspace. We calibrated it using ArUco fiducial markers and homography-based coordinate transformation. A finite state machine controls the

sequence of Detecting, Selecting, Picking, and Placing stages, using ByteTrack-based object tracking, trajectory-based pick validation, and class-informed placement. The integrated system achieved a total pick-and-place success rate of 89.6% across 48 cycles, with an operational throughput of 9 picks per minute. Explainable AI Integration for Clear Decision-Making. We applied Grad-CAM and Eigen-CAM visualization methods to confirm that the model's focus is on relevant vegetable areas, such as surface blemishes, discoloration zones, and texture issues, rather than background distractions. This enhancement improves the transparency and reliability of sorting decisions for end-users and agricultural practitioners. Interactive Real-Time Web Application Dashboard. We created a Streamlit-based web application that combines real-time detection, classification, XAI visualization, and robotic arm control into one user-friendly interface. The dashboard supports video feeds from webcams and smartphones, file uploads, annotated output downloads, live detection plots, manual servo control, and class-specific pick-and-place actions.

1.5 Project Outline

The remainder of this report is organized as follows. Chapter 2, Related Works, presents a review of previous literature, focusing on three main areas: supervised deep learning for vegetable detection and classification, semi-supervised and annotation-efficient learning in agriculture, and self-supervised learning for agricultural vision and robotics. The chapter also concludes with a summary of the research gaps that motivate the proposed framework. Next, Chapter 3, Methodology, provides a detailed description of the end-to-end system design. It includes the dataset collection strategy and annotation pipeline, image preprocessing and improvement procedures, supervised YOLO-family baseline models, and three self-supervised pretraining architectures (DINOv2-style, BYOL, and MAE). In addition, it explains the vision-guided robotic automation pipeline, which covers workspace calibration, object detection and tracking, motion planning, and the strategy for autonomous execution. This chapter also discusses the explainable AI framework, real-time web application dashboard, experimental hardware setup, and training hyperparameter configurations. Following this, Chapter 4, Results and Discussion, presents and interprets the experimental findings. It analyzes the performance of supervised YOLO baselines, evaluates the quantitative and qualitative outcomes of SSL architectures, and examines trends in loss and mAP convergence, feature space clustering, detection visualizations, confusion matrices, XAI heatmaps, and robotic arm performance metrics. It also includes a summary comparison with recent literature. Then, Chapter 5, Discussion, places these findings within the broader research landscape. It addresses system limitations such as dataset diversity, fine-grained freshness discrimination, environmental robustness, and deployment scalability. It also outlines directions for future research. Finally, Chapter 6, Conclusion, summarizes the key contributions and outcomes of the project. It reaffirms the practical viability of the proposed SSL enhanced vision-guided sorting framework and highlights potential paths

for extending the system toward large-scale commercial deployment in automated agricultural processing pipelines.

Chapter 2

Related Works

2.1 Survey of the State-of-the-art

This section reviews supervised, semi-supervised, and self-supervised approaches relevant to vegetable detection, freshness classification, and robotic agricultural automation, establishing the context for the research gaps that motivate the proposed vision-guided robotic framework for automated vegetable freshness sorting

2.1.1 Supervised Deep Learning for Vegetable Detection and Classification

Supervised deep learning remains the dominant approach in agricultural computer vision when annotated datasets are available. Khanna et al. [11] introduced FVDNet, an ensemble-based YOLOv8n model for detecting 67 fruit and vegetable classes. The model combined JensenShannon divergence with focal loss to improve small-object detection and achieved 0.82 mAP. Although a semi-supervised data annotation algorithm was used to assist labeling, the detection model itself was trained in a fully supervised manner. Similarly, Li et al. [12] proposed D,s-YOLOv10, a lightweight detection framework for tomato detection in greenhouse environments. By integrating DyFasterNet modules, D-LKA attention, and dynamic FM-WIoU loss, the model reduced parameters by 54% while achieving 91.8% mAP@0.5. This work demonstrates that supervised architectures can balance efficiency and accuracy for real-time deployment. Transformer-based supervised detectors have also been explored. Bu et al. [13] developed FVRT-DETR using a Mamba backbone and multi-scale deep feature fusion. Their model improved detection performance under occlusion and complex agricultural backgrounds, showing strong scalability for large datasets. Beyond deep learning, traditional supervised methods remain relevant. Wang et al. [14] used handcrafted texture, wavelet, and shape features combined with an SVM classifier, achieving 96.25% accuracy. Likewise, Khoiruddin et al. [15] applied MobileNetV2 for multi-class fruit and vegetable classification and reported 97.31% accuracy, demonstrating that lightweight CNN models can perform well in classification tasks. In practical measurement systems, Steinbrener et al. [16] combined Mask R-CNN and LSTM networks to estimate food volume from smartphone videos. Their supervised detection and segmentation framework achieved a mean absolute percentage error of 16.0%, showing potential for real-world applications. In summary, supervised methods achieve high detection and classification accuracy across many agricultural tasks [11, 12, 13, 14, 15, 16]. However, their performance strongly depends on large, well-annotated datasets, which are

expensive and time-consuming to construct.

2.1.2 Semi-Supervised and Annotation-Efficient Learning in Agriculture

To reduce the burden of manual annotation, semi-supervised and annotation-efficient learning methods have become increasingly important in agricultural vision. Several studies adopt pseudo-labeling and teacher student strategies. Li and Shi [17] proposed a semi-supervised DETR-based framework for plant growth monitoring, achieving competitive performance with only 5% labeled data. García et al. [18] applied pseudo-labeling and distillation techniques for vineyard segmentation and improved performance by approximately 6% using limited labeled samples. Similarly, Johanson et al. [19] introduced S3AD for small apple detection, leveraging unlabeled orchard data to enhance robustness under occlusion. More advanced pseudo-label refinement strategies were explored by Zhou et al. [20], who proposed MemoryTeacher to stabilize iterative pseudo-label updates for pest detection. Xu et al. [21] further addressed noisy annotations in UAV imagery through a teacher student framework combined with confident learning. Iterative and hybrid approaches have also shown effectiveness. Bogomasov et al. [22] reduced labeling effort by more than 60% through iterative pseudo-label refinement for fruit and vegetable detection. Deng and Lu [23], Saleh et al. [24], and Li et al. [25] demonstrated that combining pseudo-labeling with consistent regularization improves generalization across varying field conditions. Chen et al. [26] introduced preference-guided learning to improve interpretability in cropweed classification. Li et al. [27] incorporated diffusion-based generative attention to enhance robustness with sparse labeled data. Synthetic and semi-automated annotation methods also contribute to annotation efficiency. Najafian et al. [28] generated computationally annotated wheat head datasets using limited manual labels and domain adaptation. Ter-Sarkisov et al. [29] proposed a bootstrapping pipeline for livestock monitoring, iteratively expanding labeled datasets. Gomez-Zamanillo et al. [30] applied semi-supervised density estimation for pest counting, significantly reducing annotation requirements. Furthermore, Yang et al. [31] combined super-resolution and active learning to minimize annotation cost while maintaining high accuracy.

Overall, semi-supervised and annotation-efficient learning significantly reduce labeling effort and improve scalability [17, 20, 23]. Nevertheless, these methods remain sensitive to noisy pseudo-labels, confidence thresholds, and domain shifts, especially in complex agricultural environments.

2.1.3. Self-Supervised Learning for Agricultural Vision and Robotics

Self-supervised learning focuses on learning feature representations from unlabeled data before task-specific fine-tuning. This paradigm reduces dependency on manual annotations and improves feature transferability. Wang et al. [32] introduced CC-SSL, a contrastive learning framework for crop classification, achieving over 91% accuracy with limited labeled samples. Gupta et al. [33] proposed Time2Agri, which designed agriculture-specific temporal pretext tasks for crop monitoring, showing strong performance in crop mapping and yield prediction. For anomaly and quality detection, Choi et al. [34] developed CH-Rand, a color-focused self-supervised method for strawberry anomaly detection. Liu et al. [35] proposed SSC-AE, combining autoencoder reconstruction with self-supervised classification for hyperspectral defect detection. Chai et al. [36] introduced CL-ViT, which integrates self-supervised pretext tasks to improve zero-shot plant disease recognition. Additionally, Al Mamun et al. [37] provided a systematic review highlighting the growing importance of SSL in plant disease detection. Self-supervised learning has also been extended to agricultural robotics. Ci et al. [38] proposed SSL-NBV for self-supervised next-best-view planning in 3D plant reconstruction. Later, Ci et al. [39] introduced SSL-Local-NBV to improve trajectory efficiency and adaptability during robotic exploration. These works demonstrate how SSL can support real-time robotic decision making without requiring extensive ground-truth labels.

In summary, self-supervised approaches improve feature generalization and reduce reliance on labeled data [32, 33, 38]. However, their integration into real-time vegetable freshness sorting systems remains limited.

2.1.4. Research Gaps

Supervised models achieve strong performance in vegetable detection and classification [11,12]. However, they rely heavily on large annotated datasets. Semi-supervised techniques reduce labeling effort [17, 20], yet they remain sensitive to pseudo-label noise and domain shifts. Self-supervised learning further decreases annotation dependency [32, 33], but its deployment in realtime vegetable freshness sorting systems is still underexplored. Despite all this progress, there are several important problems that should be solved before an effective system for vegetable freshness detection could be developed. First, the need for large amounts of annotated data is a problem that persists in vegetable freshness detection and makes scaling to unseen vegetables expensive and tedious. Second, current approaches have not yet managed to ensure robustness in cases of different lightings, occlusions, and cluttered backgrounds. Third, there is no consensus concerning the type of learning that is best suited for agriculture and food sorting

supervised, semi-supervised, or self-supervised learning. Fourth, the use of explainable AI technologies in sorting systems is not sufficiently explored. Finally, the integration of vision with robotics is almost non-existent in the field of vegetable freshness detection.

To address these limitations, this study:

- (i.) Evaluates YOLO-based detection models for vegetable freshness classification
- (ii.) Integrates self-supervised pretraining methods such as DINO and BYOL
- (iii.) Incorporates explainable AI techniques for improved interpretability
- (iv.) Develops a real-time vision-guided robotic sorting framework

By combining representation learning, detection, explainability, and robotic execution, the proposed system provides a unified and deployable solution for automated vegetable freshness sorting.

2.2 Summary

This chapter looks at what's currently happening in computer vision and machine learning when it comes to finding vegetables, deciding how fresh they are and using robots in farming. It is divided into three ways of learning.

When we have a lot of labeled data, Supervised Deep Learning works well. Models such as FVDNet, Ds-YOLOv10 and FVRT-DETR can find vegetables accurately with some getting it right up to 91.8% of the time. Other models like MobileNetV2 can decide how fresh vegetables are with over 97% accuracy. The problem is that we need a lot of labeled data, which can be expensive to get.

Semi-Supervised and Annotation-Efficient Methods try to fix this problem by using labels working with other models and making fake data. These methods can work well with very little labeled data, sometimes as low as 5% and can reduce the time it takes to label data by over 60%. However they can still make mistakes if the fake labels are wrong or if the conditions in the field are very different.

Self-Supervised Learning goes further by learning from data that has not been labeled at all. Some approaches like CC-SSL, Time2Agri and CL-ViT are very good at finding features in the data. Others like SSL-NBV and SSL-Local-NBV can even help robots decide what to do.. Using this in real-time to sort fresh vegetables is still not very common.

There are some problems that still need to be solved such as not having enough labeled data, not being able to work well in different lighting conditions, not knowing which learning method is best, farming not using explainable AI much and not combining vision systems with robots very well.

To solve these problems the proposed system combines finding vegetables using YOLO learning from data using DINO and BYOL explainable AI and a real-time robotic sorting pipeline all into one system that can be used in the real world.

Chapter 3

Materials and Method

3.1 Materials

The overall working procedure of this project in Figure 3.1 involves a sequential pipeline which starts by understanding the dataset. The dataset was manually collected within 8 different vegetables, 16 subclasses and one multilabel that includes other 16 subclasses with caution and then annotated with a roboflow annotator. The dataset is separated in train, test and valid subsets. After that, we will preprocess the dataset to resolve any corrupt image, misclassification, false annotations and the augmentation and then an object detection models three different versions will be applied which includes YOLOv12, YOLOv11, YOLOv10 model to find the best performing backbone and XAI for interpretation. Then, will apply three Self Supervised Learning methods to analyze and validate the models performance through evaluation, clustering and loss curve analysis. Also, using the best performing SSL method, we will build a streamlit web application dashboard over the SSL models and detection functionalities for vegetables. Finally, a robotic arm will be built to create a fully automated system to perform pick-and-place whenever a particular vegetable is in range of a standby mobile camera.

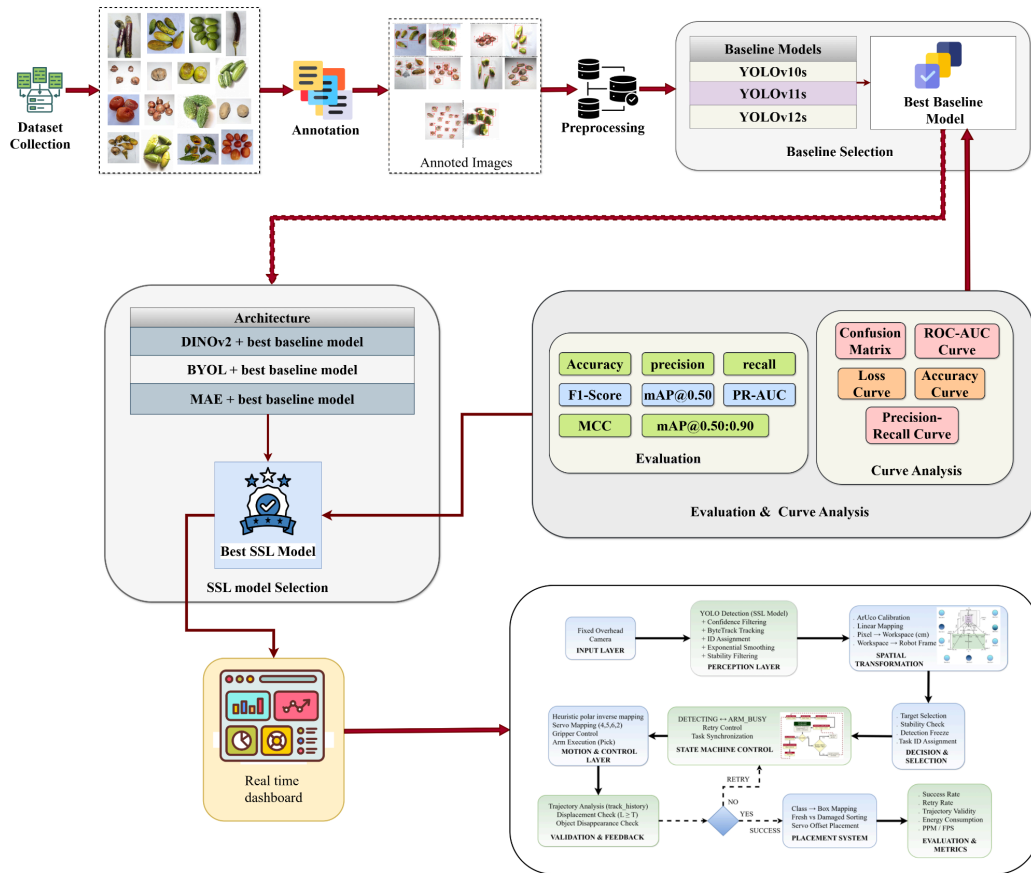


Figure 3.1: Workflow Diagram

3.1.1 Dataset Collection SetUp

The vegetable dataset was obtained using four different Android phones Google Pixel 9, Samsung Galaxy S23, Vivo Y22s, Apple iPhone 15 Pro Max, and Samsung Galaxy A55 to ensure diversity in imaging conditions and quality of capture. The images were captured in front of a big white art paper background to ensure comparable contrast and minimize visual noise. The vegetables were posed both sparsely and in a structured fashion to incorporate spatial variability, and images were taken in two visual states: fresh and damaged. This setup allowed controlled but varied image acquisition, enhancing the dataset's robustness and generalizability to future multi-class classification and computer vision tasks.

Table 3.1: Specifications of Mobile Devices Used for Image Acquisition

Device	Display	Rear Camera	Processor	RAM/Storage
Google Pixel 9	6.3" AMOLED, 1080×2400	50 MP + 12 MP	Tensor G3	8 GB, 128/256 GB
Samsung Galaxy S23	6.1" AMOLED 2X, 1080×2340	50 MP + 12 MP + 10 MP	Snapdragon 8 Gen 2	8 GB, 128/256 GB
Vivo Y22s	6.55" IPS LCD, 720×1612	50 MP + 2 MP	Helio G85	4 GB, 128 GB
Samsung Galaxy A55	6.4" Super AMOLED, 1080×2400	64 MP + 12 MP + 5 MP + 5 MP	Exynos 1380	6 GB, 128 GB
iPhone 15 Pro Max	6.7" OLED, 2796×1290	48 MP + 12 MP + 12 MP	A17 Pro	8 GB, 256 GB

3.1.2 Dataset Collection Strategy

To develop an effective multi-class vegetable classification and detection dataset, we collected images of eight types of non seasonal vegetables like cucumber, tomato, bitter gourd, potato, pointed gourd, onion, eggplant, and lemon in two visual conditions: fresh

and damaged. This created sixteen sub-classes, each represented in an orderly manner to ensure balance within the dataset. Table 3.2 shows the dataset collection distribution for this work. At First, Single-class images were captured first to represent individual vegetable conditions on their own. 50 images per sub-class were collected, totaling 800 single-class images. There was only one instance of the vegetable per image, photographed against controlled lighting and neutral backgrounds to minimize visual noise. Afterwards, to promote intra-class variation, single-class combinational images were collected in scattered and structured arrangements. 105 scattered and 105 structured images were captured for each sub-class, where multiple instances of the same sub class were randomly or structurally arranged, respectively. These images represented spatial distribution variation while retaining single-class purity, yielding 210 combinational images per sub-class. Then, the multi-label subset was prepared to simulate real-world classification and detection problems with co-occurring vegetable types. Vegetables were paired in four pre-decided pairs: (tomato + cucumber), (bitter melon + pointed melon), (potato + onion), and (brinjal + lemon). There were images of fresh and damaged conditions of both the vegetables in each pair, thus four sub-classes for each pair. For each pair, 120 multi-label images were collected, divided evenly into 60 dispersed and 60 structured configurations. In every multi-label image, all four sub-classes within a pair co-occurred together. To add variation, dominance was switched between sub-classes within a pair: some images had one sub-class in higher numbers (five to six instances) and the others in lower numbers (two to three instances). Finally, the total dataset comprises 4,640 images in all collection methods in a perfectly balanced manner across all 16 sub classes. Each vegetable pair equally contributed to the multi-label subset with 120 images per pair. This arrangement allows for both intra-class variability (fresh vs damaged) and inter-label challenge (multi-label co-occurrence), making the dataset suitable to train robust classification and detection models with high accuracy.

Table 3.2: Data Collection Strategy Distribution

Dataset Category	Subset / Pair	Per Sub-Class	Total Sub-classes	Arrangement	Total Images
Single-class	Isolated	50	16	Individual vegetables	800
Single-class Combinational	Scattered	105	16	Random placement of multiple instances	1,680
Single-class Combinational	Organized	105	16	Structured rows/clusters	1,680

Multilabel	Tomato + Cucumber	120	4	60 Scattered + 60 Organized	120
Multilabel	Bitter Gourd + Pointed Gourd	120	4	60 Scattered + 60 Organized	120
Multilabel	Potato + Onion	120	4	60 Scattered + 60 Organized	120
Multilabel	Eggplant + Lemon	120	4	60 Scattered + 60 Organized	120
Total					4,640

3.1.3 Dataset Description

After the strategy, we tried to collect the same number of images. Due to some image file error, the roboflow removed some corrupted image files. For this work, the dataset is well balanced for both classification and annotated detection type analysis. The classification images are divided into single class and multiclass images. The classification single class images are distributed in 8 different classes where each has two types of subclass (Fresh and Damaged) shown in Table 3.3 and multilabel images in quadrates in a single folder with a total of 541 images. Finally, the annotated images for detection are distributed into train 70%, test 15% and validation 15% merging both single class and multi-labels images.

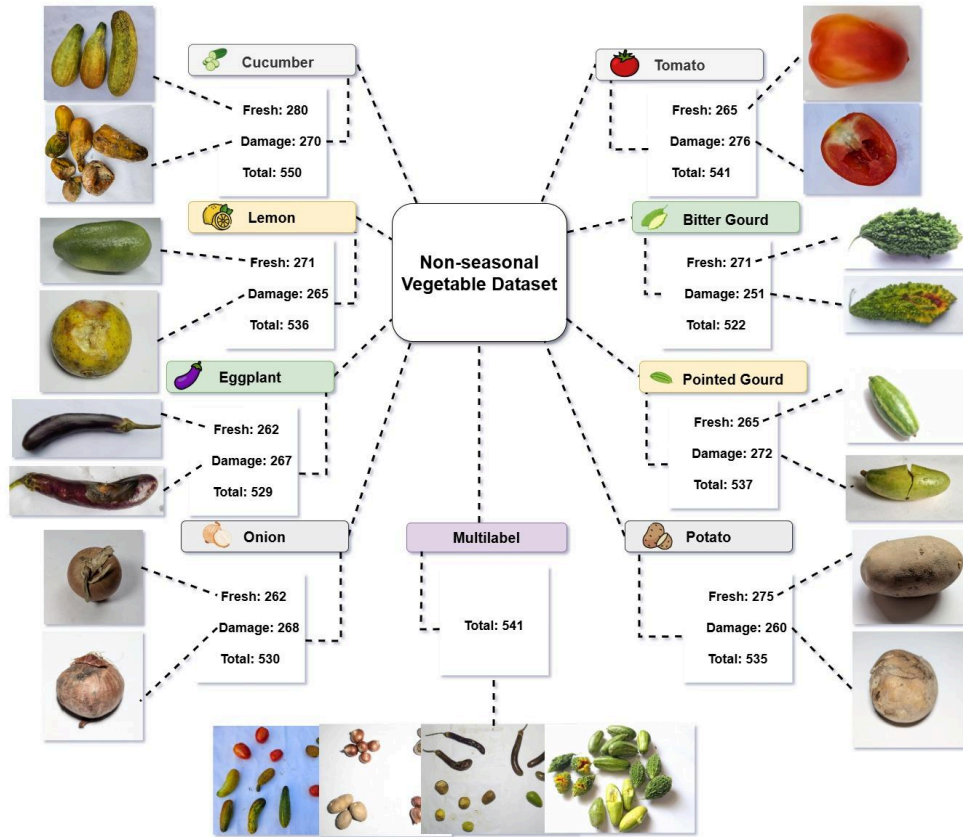


Figure 3.2: Dataset Workflow

The single class image number is shown in Table 3.3, Figure 3.2 and Figure 3.3 which are more or less almost the same in numbers like the strategy that has been made. After taking more or less the same number of images per subclass, that makes 4270 in total. Then we got 531 in the total number of multilabel images. Moreover, Figure 3.4 and Figure 3.5 shows the histogram of the dataset to visualize that the dataset is well balanced throughout the distribution and some dataset visualizations..

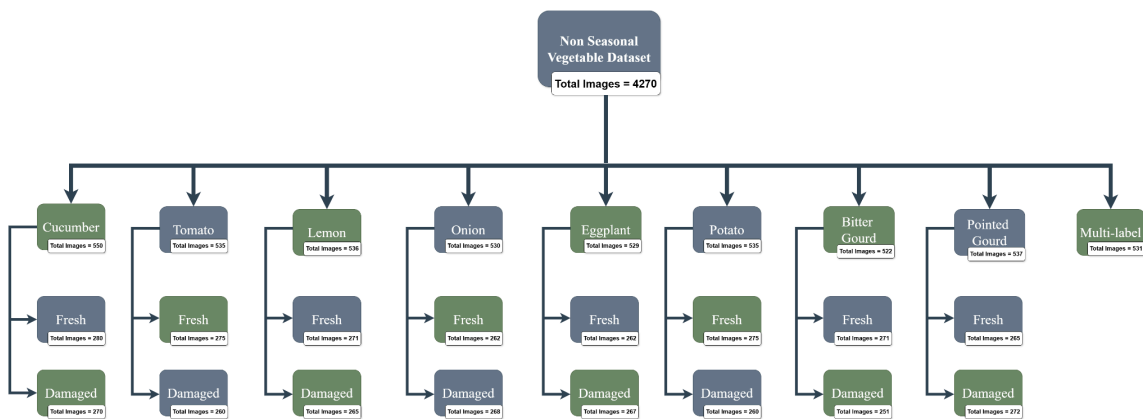


Figure 3.3: Classwise distribution Flow

Table 3.3: Dataset distribution for Single Class Classification images

Class	Fresh	Damaged	Total
Cucumber	280	270	550
Tomato	275	260	535
Bitter Gourd	271	251	522
Pointed Gourd	265	272	537
Potato	275	260	535
Onion	262	268	530
Eggplant	262	267	529
Lemon	271	265	536
Total			4270

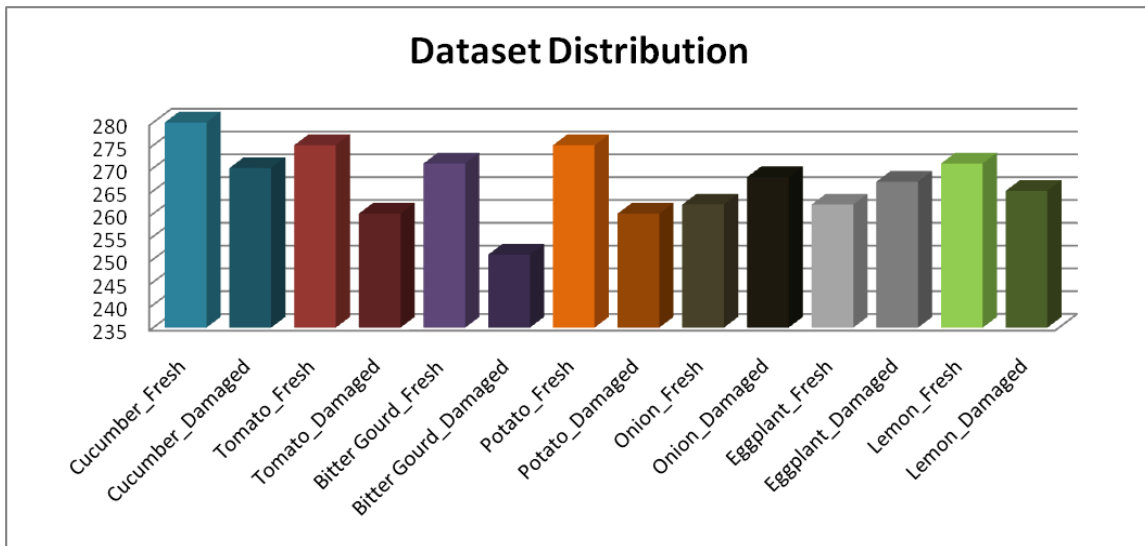


Figure 3.4: Dataset Distribution for Object Detection



Figure 3.5: Visualized Images

3.1.4 Dataset Annotation and Processing

After the collection, all collected images were standardized before training to ensure standard quality, filter between the healthy and corrupted images, augmentation to improve model performance. Each image was resized to 640x640 pixels. One simple augmentation technique will be applied named horizontal flip because the image is collected in a way that no other augmentation is required to train the model. After that the dataset was divided into 70% training, 15% validation and 15% testing sets, ensuring class balance for both fresh and damaged vegetable samples after the annotation on roboflow.

In addition to preprocessing, the dataset was annotated collaboratively using the Roboflow platform. Each participant took on specific annotation responsibilities, allowing the work to be divided fairly and ensuring that the labels remained consistent across all categories. The annotation process focused on bounding box labeling for both fresh and damaged vegetable samples. The overall contributions are presented in Table 3.4, which records how the tasks were distributed among participants and reflects the collective effort behind preparing the dataset. This organized approach not only made the annotation process more efficient but also provided clarity and transparency regarding

each participant's role. After the annotation, the dataset version was formatted into YOLO(images and labels) shown in Figure 3.6.

Table 3.4: Distribution of Dataset Annotation

No.	Class	Numbers Of Images
1	Bitter Gourd Fresh	271
	Bitter Gourd Damaged	251
	Pointed Gourd Fresh	265
	Pointed Gourd Damaged	272
	Multilabeled	135
2	Tomato Fresh	265
	Tomato Damaged	271
	Cucumber Fresh	280
	Cucumber Damaged	270
	Multilabeled	132
3	Potato Fresh	273
	Potato Damaged	260
	Onion Fresh	262
	Onion Damaged	268
	Multilabeled	132
4	Eggplant Fresh	262
	Eggplant Damaged	267
	Lemon Fresh	264
	Lemon Damaged	265
	Multilabeled	132

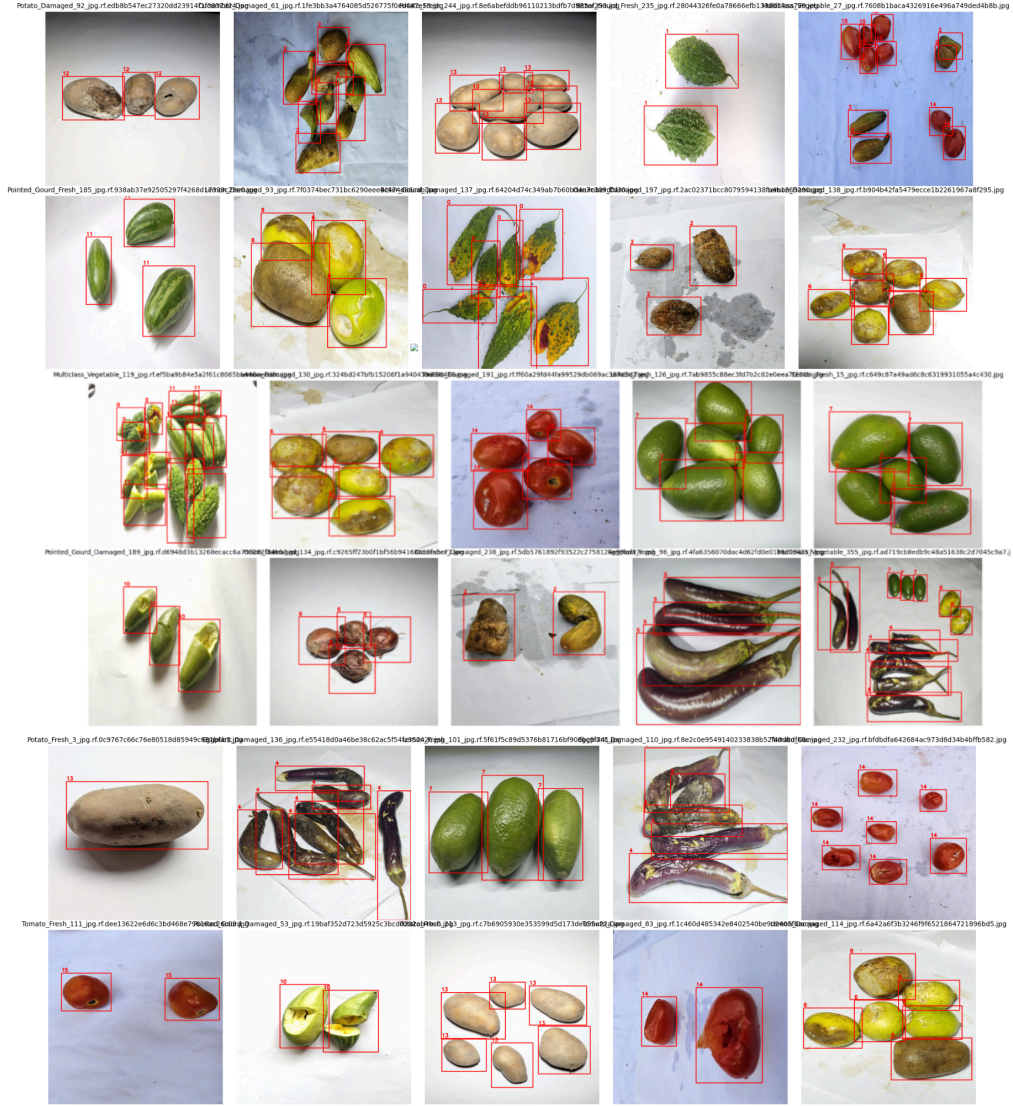


Figure 3.6: YOLO Formatted Annotated Images

3.1.5. Experimental Setup

The experimental setup is organized into two main subsections:

- **Hardware Setup:** This section describes the overall workspace and the components used in the system.
- **Training Configuration:** This section outlines the model training procedure, including the selected hyperparameters.

3.1.5.1. Hardware Setup

The designed system involves integrating the vision-guided robot manipulator framework used to detect and sort vegetables automatically. The Hiwonder xArm 1S robot manipulator, a fixed RGB monocular camera, and a structured workspace are included in the design. In particular, the Hiwonder xArm 1S is by nature a robot manipulator with six

DoF and the Table 3.5 shows the overall specification of the arm. However, for simplicity purposes in controlling the robot, only four DoF have been considered in this experiment, whereas the other joints have been fixed. Here, the active joints include base rotation, arm extension, vertical movement, and gripping joints. These joints are enough for carrying out pick-and-place movements of vegetables. For stability purposes, the robot arm has been attached to a wooden board and the top-down camera is mounted on an adjustable stand that allows having a constant workspace for the experiment. Besides, fiducial markers have been considered as part of the camera calibration process to allow the transformation of images into a real world space coordinate system. Then, the detected object locations undergo linear mapping to convert them into robot coordinates. Then, an inverse kinematics control method is used for controlling the robot. This is followed by performing sequential pick and place operations through USB control based servo-positioning where object grasping operations are done by a parallel gripper at specified open and closed positions. This way, object manipulation based on object detections can be performed in real time.

Table 3.5: Specifications of the Hiwonder xArm 1S Robotic Arm

Parameter	Specification
Model	Hiwonder xArm 1S
Degrees of Freedom (DoF)	6 DOF (4 DOF used)
Actuation	Servo Motor-based
Control Interface	USB Serial Communication
Working Voltage	12V DC
Workspace Size	~28 cm $\times$ 14 cm
Base Rotation (Servo 6)	200–700
Arm Extension (Servo 5)	275–520
Vertical Motion (Servo 4)	400–950
Gripper Type	Servo-based parallel gripper

Gripper Range	100 (open) – 510 (closed)
Motion Control	Position-based (PWM control)
Average Pick Time	~1–3 seconds
Camera Setup	Fixed top-down RGB camera
Mounting Structure	Wooden base + adjustable stand

3.1.5.2. Training Configuration

All experiments were conducted under a uniform training regimen to ensure fair comparison across different configurations. Training was performed on the Kaggle GPU environment equipped with an NVIDIA Tesla T4 (15 GB VRAM), 4 CPUs, and 31 GB RAM, using PyTorch 2.6.0 with CUDA 12.4 support and the Ultralytics YOLO framework (v8.3.232). The vegetable freshness dataset contained sixteen categories, representing fresh and damaged states across multiple vegetable types, and all images were resized to 640×640 pixels before training. Two paradigms were examined: self-supervised pipelines (DINOv2-style and BYOL encoders) and supervised YOLO baselines (YOLOv10-s, YOLOv11-s, YOLOv12-s). Pretrained SSL weights were reused to initialize YOLO detectors, ensuring fair comparison. Training was conducted for 50 epochs with a batch size of 12, learning rate 0.01, momentum 0.937, and weight decay 0.0005, followed by a 3-epoch warmup where bias learning rate was set to 0.1 and momentum gradually increased to 0.8. Detection employed a confidence threshold of 0.25 and IoU thresholds of 0.50- 0.95, while SSL models fixed the temperature parameter at 0.1 for stable similarity estimation. Table 3.6 presents the supervised YOLO baselines. All three models adopt AdamW (auto) as the optimizer, with identical batch sizes (12) and image sizes (640×640). Training spans 50 epochs, with learning rates automatically tuned around 0.002. Weight decay is consistently set at 0.0005, and a warmup of 3 epochs is applied. Momentum remains fixed at 0.9, with no gradient accumulation. Structural differences emerge in hidden dimensions: YOLOv12-s integrates approximately 464 fused layers, YOLOv11-s uses 203 layers, while YOLOv10-s remains model-defined. Despite these variations, all three models employ BCE + DFL for layer definitions, disable explicit loss functions (marked as False), and support multi-scale training and mixed precision (AMP). Pretrained weights differ only in version-specific

checkpoints (yolo12s.pt, yolo11s.pt, yolo10s.pt). Analytically, these similarities underscore the stability of YOLOs training pipeline, while the differences in hidden dimensions reflect incremental architectural refinements across versions.

Table 3.6: Comparative Hyperparameter Settings for Baseline Models

Hyperparameters	YOLOv12-s	YOLOv11-s	YOLOv10-s
Optimizer	AdamW (auto)	AdamW (auto)	AdamW (auto)
Batch Size	12	12	12
Image Size	640	640	640
Epochs	50	50	50
Learning Rate	auto (~ 0.002)	auto (~ 0.002)	auto (~ 0.002)
Weight Decay	0.0005	0.0005	0.0005
Warm-Up	3 epochs	3 epochs	3 epochs
EMA / Momentum	Momentum = 0.9	Momentum = 0.9	Momentum = 0.9
Gradient Accumulation	0	0	0
Hidden Dim / Channels	≈ 464 fused layers	≈ 203 layers	Model-defined
Loss Function	BCE + DFL	BCE + DFL	BCE + DFL
Multi-Scale Training	False	False	False
Mixed Precision (AMP)	True	True	True
Pretrained Weights	yolo12s.pt	yolo11s.pt	yolo10s.pt

Table 3.7 outlines the distinct design choices between the two self-supervised pipelines. DINOv2 employs a larger hidden dimension (2048) and Softmax activation, reflecting its emphasis on high-capacity feature extraction, while BYOL uses a smaller hidden dimension (512) with ReLU, favoring efficiency. DINOv2 integrates teacher and student temperature parameters (0.07 and 0.1) to stabilize similarity estimation, whereas BYOL

omits these but applies a stricter confidence threshold (0.7). Both share commonalities such as AdamW optimizer, 50 epochs, and EMA = 0.996, yet their divergence in loss functions Cross-Entropy for DINOv2 versus Mean Squared Error for BYOL illustrates differing philosophies in representation learning. These contrasts highlight how DINOv2 prioritizes structured feature alignment, while BYOL emphasizes smoother embedding consistency.

Table 3.7: Comparative Hyperparameter Configuration Across SSL Architectures

Hyperparameter	DINOv2 Style	BYOL
Optimizer	AdamW	AdamW
Backbone Batch Size	32	64
Fine-Tune Batch Size	12	12
Feature Image Size	224	165
Fine-Tune Image Size	640	640
Grad. Accumulation	2	-
Activation Function	Softmax	ReLU
Hidden Dimension	2048	512
Epochs	50	50
Confidence Threshold	-	0.7
Teacher Temperature	0.07	-
Student Temperature	0.1	-
Learning Rate	5e-4	5e-4
Weight Decay	0.05	0.05
Loss Function	Cross-Entropy	Mean Squared Error
EMA	0.996	0.996
Warmup Fraction	0.1	-
Centering Momentum	0.9	-

To improve generalization, RandAugment was applied during SSL pretraining, and

YOLO fine-tuning incorporated mosaic augmentation, HSV jitter ($h=0.015$, $s=0.7$, $v=0.4$), random translation (0.1), random erasing (0.4), and horizontal flips ($p=0.5$). These augmentations enhanced robustness against lighting, occlusion, and background clutter. Final detectors were evaluated using the best validation checkpoint, ensuring stable and reliable performance reporting.

3.1.6. Evaluation Matrices

To maintain consistency and rigor in performance assessment, this study utilized a set of evaluation metrics that collectively address both classification accuracy and localization precision key factors in agricultural object detection, particularly for freshness classification. Given the challenge of distinguishing between healthy and damaged vegetable states, the following metrics were selected:

- **Precision:** Precision reflects the proportion of correctly predicted fresh or damaged vegetable instances out of all positive predictions. In freshness detection, this metric helps reduce false positives, such as misclassifying background textures or non-vegetable elements as produced. It plays a vital role in downstream tasks like automated sorting and shelf-life estimation by Equation 1.

$$Precision = \frac{TP}{TP+FP} \quad (1)$$

where TP and FP denote true positives and false positives, respectively.

- **Recall:** Recall indicates how effectively the model identifies actual instances of fresh or damaged vegetables. A high recall ensures minimal false negatives, which is crucial for applications like spoilage tracking or yield analysis. The metric is defined in Equation 2.

$$Recall = \frac{TP}{TP+FN} \quad (2)$$

with FN representing false negatives.

- **Mean Average Precision (mAP):** To evaluate detection performance across varying confidence thresholds, two mAP variants were reported:

-mAP@50: Measures average precision at a fixed IoU threshold of 0.5. This relatively lenient metric is widely used in agricultural detection benchmarks. It often yields higher scores and serves as a common baseline in agricultural detection studies. In freshness detection tasks, mAP@50 helps capture general detection capability without heavily penalizing minor localization errors.

-mAP@[50:90]: Computes average precision across multiple IoU thresholds (from 0.5 to 0.9 in 0.05 increments), offering a more stringent assessment of localization accuracy. It gives a stricter way to check how well models match object boundaries. For vegetable freshness detection, mAP@[50:90] is especially useful because it tests the model's strength under tougher conditions. This helps in

spotting small or subtle damaged areas more reliably.

- **Intersection over Union (IoU):** IoU serves as the core criterion for bounding box evaluation, quantifying the overlap between predicted and ground truth annotations. It is calculated as Equation 3.

$$IoU = \frac{Area\ of\ Overlap}{Area\ of\ Union} \quad (3)$$

A prediction is considered valid if its IoU exceeds the designated threshold.

For system-level evaluation, detection stability is used to measure the consistency of detections across frames, as defined in Equation 4.

$$S_d = \frac{N_{stable}}{N_{detections}} \quad (4)$$

In contrast, tracking loss rate captures the robustness of object tracking by quantifying lost tracks, as shown in Equation 5.

$$R_{loss} = \frac{N_{lost}}{N_{detections}} \quad (5)$$

To evaluate manipulation performance, pick success rate is used to measure the ratio of successful pick-and-place operations, as defined in Equation 6.

$$S_p = \frac{N_{success}}{N_{cycles}} \quad (6)$$

Furthermore, the average task completion time represents the mean duration required per successful operation, as expressed in Equation 7.

$$T_{avg} = \frac{1}{N_{success}} \sum_{i=1}^{N_{success}} T_i \quad (7)$$

To capture system robustness, retry rate is used to measure the frequency of repeated attempts due to failures, as defined in Equation 8.

$$R_{retry} = \frac{1}{N_{cycles}} \sum_{i=1}^{N_{cycles}} r_i \quad (8)$$

Additionally, throughput, expressed as picks per minute (PPM), evaluates operational efficiency, as shown in Equation 9.

$$PPM = \frac{N_{success}}{60} \quad (9)$$

Energy consumption is estimated to assess system efficiency during operation, as defined in Equation 10.

$$E = \sum_{i=1}^{N_{success}} V_i \cdot I \cdot T_i \quad (10)$$

Finally, trajectory consistency is used to measure the stability of object motion across frames, as defined in Equation 11, while the trajectory validity condition is enforced as shown in Equation 12. In addition, disappearance validation rate evaluates successful object removal based on tracking disappearance, as shown in Equation 13.

$$L_i = \sum_{k=1}^n \| p_i^k - p_i^{k-1} \| \quad (11)$$

$$L_i > \tau_{traj}, \quad \tau_{traj} = 15 \quad (12)$$

$$R_{disp} = \frac{N_{disappear}}{N_{cycles}} \quad (13)$$

3.2. Method

3.2.1. Baseline Supervised Detection Model

We employ three state-of-the-art supervised object detection models from the YOLO family to evaluate vegetable freshness detection. This section provides model descriptions, architectural highlights, and how they compare in relation to this work.

— YOLO Series (YOLOv10s, YOLOv11s, YOLOv12s)

All YOLO models follow a one-stage detection paradigm, predicting bounding boxes and class probabilities in a single forward pass for high-speed inference. All three variants were trained under identical conditions on our vegetable freshness dataset spanning sixteen subcategories across eight vegetable types, using COCO-pretrained weights as initialization to leverage transfer learning before domain-specific fine-tuning.

- **YOLOv10s:** A lightweight single-stage baseline offering fast inference with comparatively lower precision, exhibiting reduced localization accuracy under strict IoU

constraints, making it better suited to scenarios where inference speed is prioritized over fine-grained spatial accuracy.

- **YOLOv11s:** An improved variant featuring an enhanced backbone and attention mechanisms that deliver a more balanced precision recall trade-off, demonstrating smoother loss convergence and stronger generalization in early training epochs compared to YOLOv10s.
- **YOLOv12s:** The most advanced of the three, incorporating area-attention modules and FlashAttention to enable larger receptive fields with reduced computational overhead, achieving the highest detection accuracy and localization precision across all evaluated metrics.

Including all three YOLO variants enables a structured comparison of accuracy, inference speed, and convergence stability under identical training conditions. YOLOv10s is particularly attractive for latency-sensitive deployments due to its lightweight design, whereas YOLOv12s demonstrates superior adaptability under occlusion, multi-instance arrangements, and subtle freshness variation. Overall, architectural progressions such as attention module design, parameter count, and the interplay between receptive field size and computational cost significantly shape detection performance in cluttered vegetable sorting environments.

3.2.2. Self Supervised Learning (SSL) Architecture

Self-supervised learning (SSL) enables representation learning from unlabeled data by constructing surrogate tasks that guide the model to extract meaningful visual features without manual annotations. Recent advances such as DINOv2, BYOL, and MAE have demonstrated state-of-the-art representation quality across diverse vision benchmarks. In this study, all three are integrated with YOLO backbones to enhance feature representation prior to fine-tuning for vegetable freshness detection.

3.2.2.1. DINOv2-Style Pretraining

The DINOv2-style framework, illustrated in Figure 3.7, incorporates a multi-crop strategy, generating two global crops and eight local crops from each input image. These augmented views are simultaneously processed by a teacher–student pair of YOLO backbones:

- Both networks are followed by projection heads. The teacher network applies centering and softmax normalization to stabilize output distributions, while the student network applies only softmax.
- The training objective is to align student outputs with teacher outputs across all crop views, enforcing augmentation invariance in the learned feature space.
- The teacher network is updated via exponential moving averaging (EMA) of the

student weights, while the student is updated through standard gradient descent.

After pretraining, the learned YOLO backbone weights are transferred to the detection framework. A new YOLO neck and detection head are initialized from scratch and fine-tuned on the labeled vegetable freshness dataset for class-specific localization and classification across sixteen subcategories.

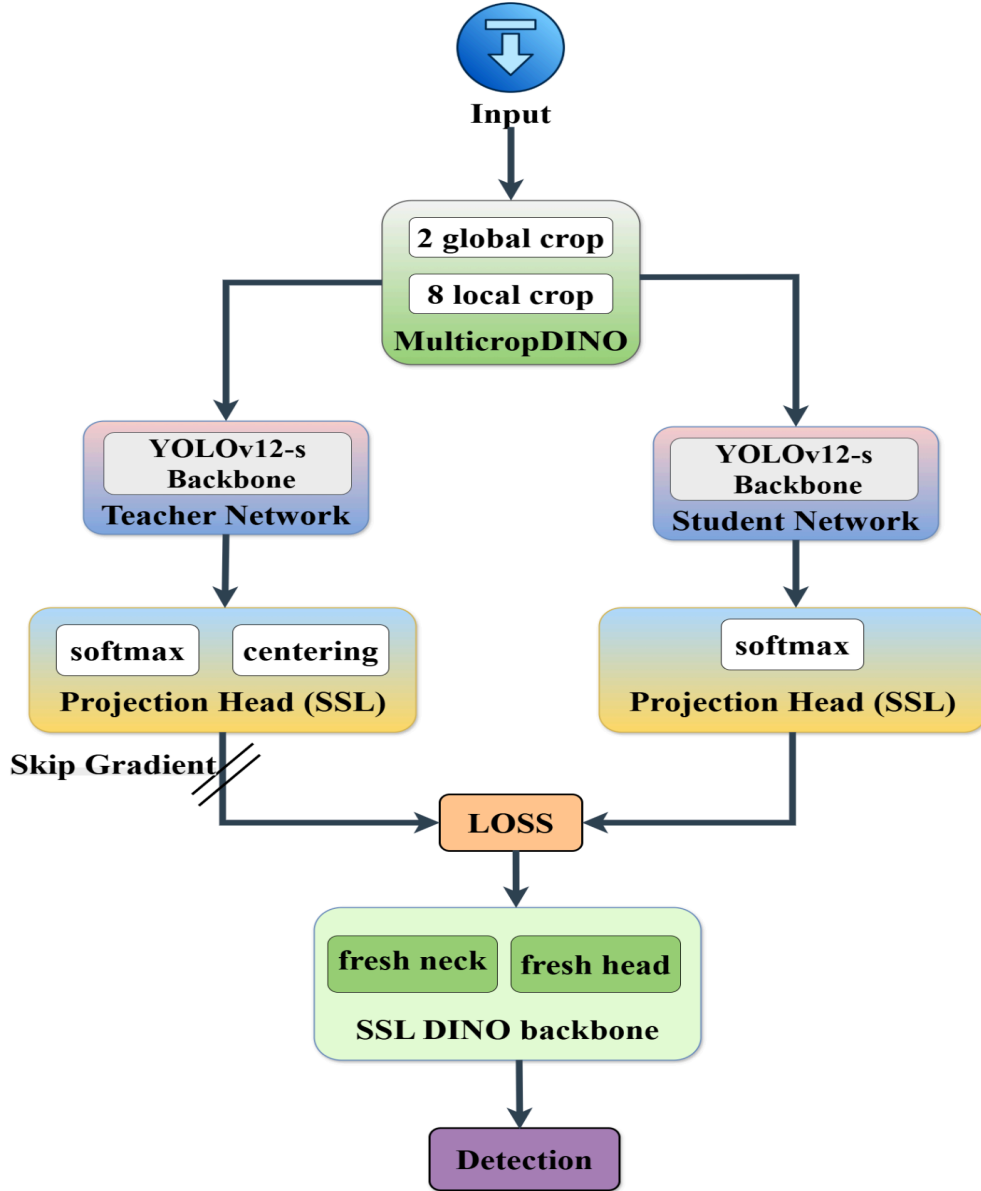


Figure 3.7: DINOv2 Style Architecture

3.2.2.2. BYOL-Style Pretraining

The BYOL (Bootstrap Your Own Latent) mechanism, shown in Figure 3.8, is embedded within the YOLO backbone and neck to enable self-supervised pretraining without negative pairs:

1. Each input image is augmented into two distinct views and independently processed by an online network and a target network sharing the same YOLO backbone architecture.
2. Feature hooks capture intermediate outputs from the YOLO backbone and neck, prior to the detection head. These feature maps are globally pooled into fixed-length latent vectors, which are then passed through projection and prediction heads.
3. The online network is trained to predict the target network's projected representations using cosine similarity loss. The online network is updated via gradient descent, while the target network is updated through exponential moving averaging (EMA) of the online weights, eliminating the need for contrastive negative samples.
4. After pretraining, the backbone and neck weights are transferred to the YOLO detection framework, while the detection head is initialized and trained from scratch on labeled data.

After pretraining, the transferred backbone and neck weights initialize the detection model, providing robust feature representations that improve localization stability under varying lighting conditions, occlusion, and spatial configurations encountered in the vegetable dataset.

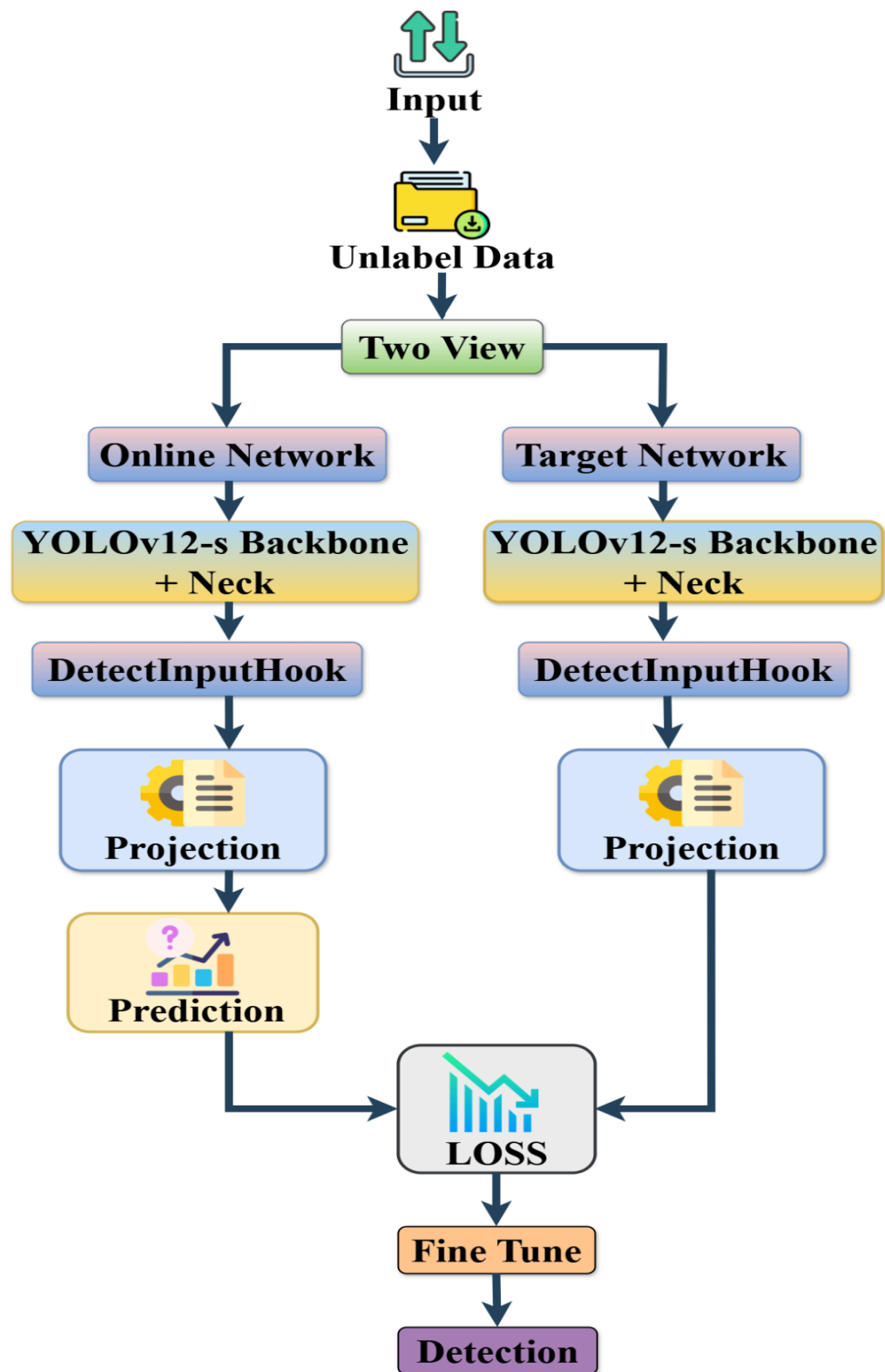


Figure 3.8: BYOL Architecture

3.2.2.3. MAE-Style Pretraining

The MAE (Masked Autoencoder) framework, depicted in Figure 3.9, adopts a reconstruction-based pretraining strategy by masking a significant proportion of input image patches and training the model to recover the missing content:

1. Each input image is divided into a fixed grid of non-overlapping patches. A masking ratio of 75% is applied, randomly removing the majority of patches while retaining only the visible minority as encoder input.
2. The visible patches are processed by the YOLO backbone acting as the encoder, producing latent representations that capture the structural and semantic content of the unmasked regions, as shown in Figure 9.
3. A lightweight decoder receives the encoded visible tokens alongside mask tokens representing the absent patches. The decoder reconstructs the full image in pixel space, forcing the encoder to learn context-dependent, shape-aware, and texture-aware embeddings rather than relying on local pixel redundancy.
4. After pretraining, the decoder is discarded and the pretrained YOLO backbone encoder weights are transferred to the detection framework. The YOLO neck and detection head are initialized from scratch and fine-tuned on the labeled vegetable freshness dataset.

This reconstruction objective encourages the backbone to develop globally coherent feature representations, improving robustness to occlusion, background clutter, and illumination variation conditions commonly encountered in multi-instance vegetable sorting environments.

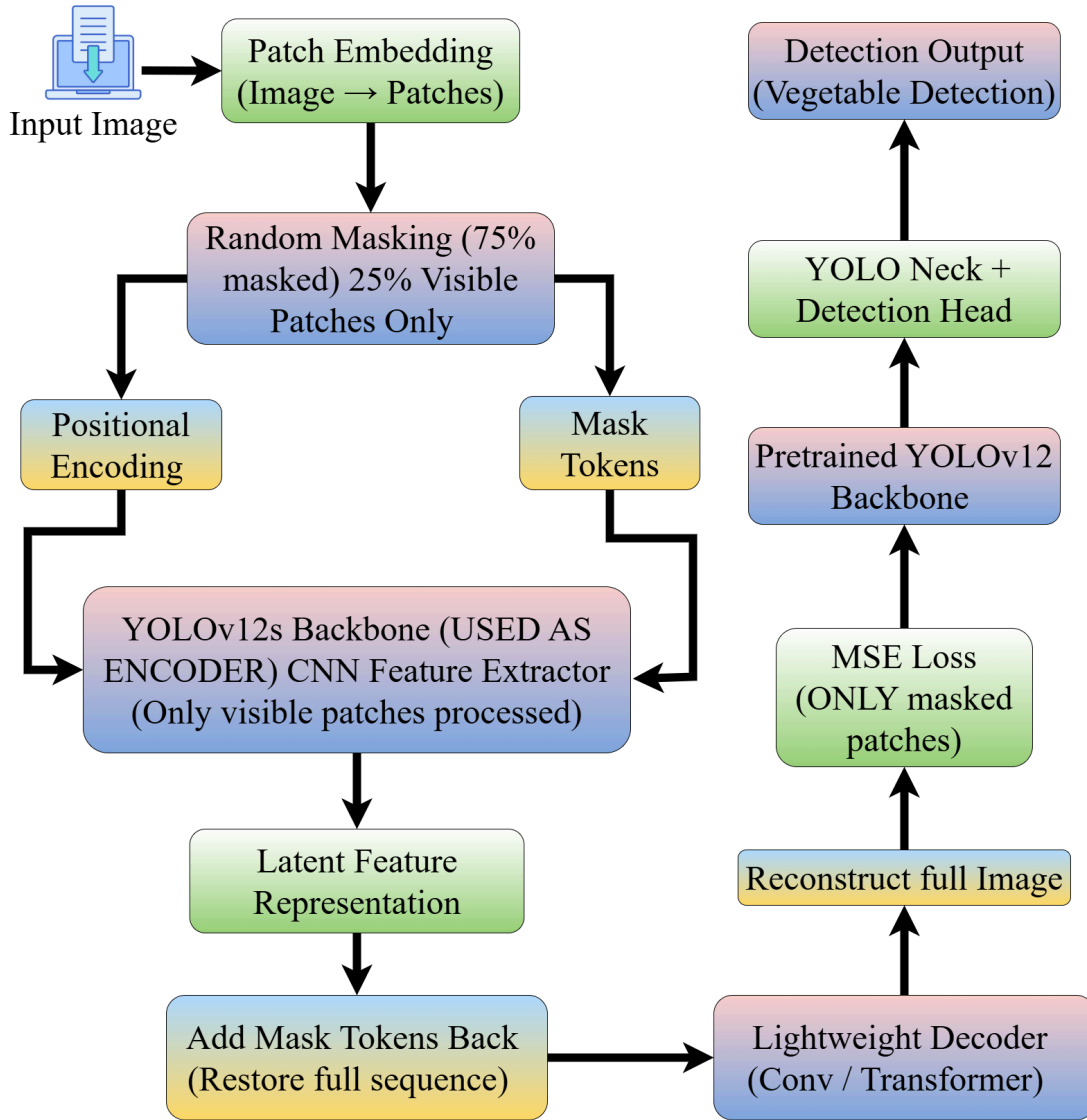


Figure 3.9: MAE Architecture

3.2.3. Explainable AI (XAI) Framework

In order to improve the interpretability of the deep learning detection model that is proposed, Explainable Artificial Intelligence techniques are also considered. The use of such techniques gives visual understanding of the reasoning behind model decisions by considering that decisions are made based on relevant and meaningful features. In the present research study, two visualization techniques based on gradient calculations, such as Grad-CAM and Eigen-CAM are used.

Grad-CAM (Gradient-weighted Class Activation Mapping): Grad-CAM is used for creating class-specific localization maps by taking advantage of the gradients of the target class concerning the last convolutional layer. More precisely, importance scores are

calculated via global average pooling of the gradients, which are subsequently multiplied by the feature maps to yield a heatmap. The heatmap marks the regions in the input image that are important for the classifier's decision-making process. For instance, in the task of detecting vegetable freshness, Grad-CAM is effective at highlighting essential features such as discolorations, textural anomalies, and damage zones.

Eigen-CAM: Eigen-CAM is used as one such gradient-based method which makes use of principal component analysis (PCA) to obtain activation maps from the convolution layer feature maps. While the other methods rely on gradients of the model outputs, Eigen-CAM obtains the prominent spatial features by analyzing the activation maps without using the gradients. Hence, the obtained results will be more stable and robust to noise as compared to other methods. Eigen-CAM gives a global representation of the most relevant regions of the image that help in classification of images.

Overall, the use of Grad-CAM and Eigen-CAM improves the transparency of the presented method by ensuring explanations from both a gradient perspective and a features-based perspective. By combining both perspectives, the predictions generated using the method become humanly understandable, accurate, and reliable.

3.2.4. Vision-Guided Robotic Automation Pipeline

3.2.4.1. System Architecture Overview

As seen in Figure 3.10, the proposed system employs a closed-loop robotic manipulation vision approach consisting of four stages: perception, transformation, control, and validation. The vision-based approach uses an overhead camera to capture images of the work environment, where the image is analyzed using a self-supervised detection model called YOLO. Then, object

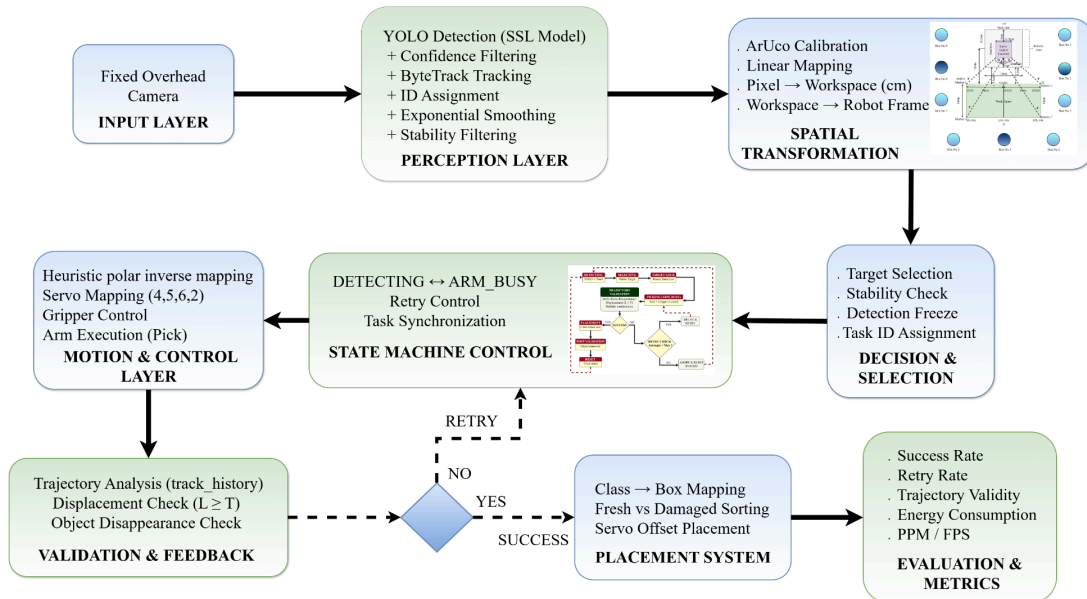


Figure 3.10: System Architecture Overview

coordinates are translated to the robot's workspace from the image plane through ArUco homography, matching the robot coordinates. Target selection via freezing detection is followed by applying inverse kinematics to convert the spatial coordinates into movement commands to pick up the objects. For validation purposes, the success of picking is checked in terms of the objects' movement and disappearance; otherwise, the procedure is re-attempted. Lastly, the object is deposited in a predetermined location depending on the category of the object. The whole process runs on a state machine basis, with ongoing performance assessment.

3.2.4.2. Workspace Calibration and Camera Coordination

The calibration of the workspace and the correlation between the camera and the robot are performed with the help of a geometric configuration based on the use of planar markers, as depicted in Figure 3.11. Four fiducial markers (ArUco) have been positioned at the corners of the workspace, forming a rectangular working space of dimensions 28 cm $\times$ 14 cm. As indicated in the Figure, the coordinate system is set up such that the origin coincides with Marker 1, or (0, 0), the +X direction corresponds to the line from Marker 1 to Marker 2, and the -Y direction coincides with the line from Marker 1 to Marker 4.

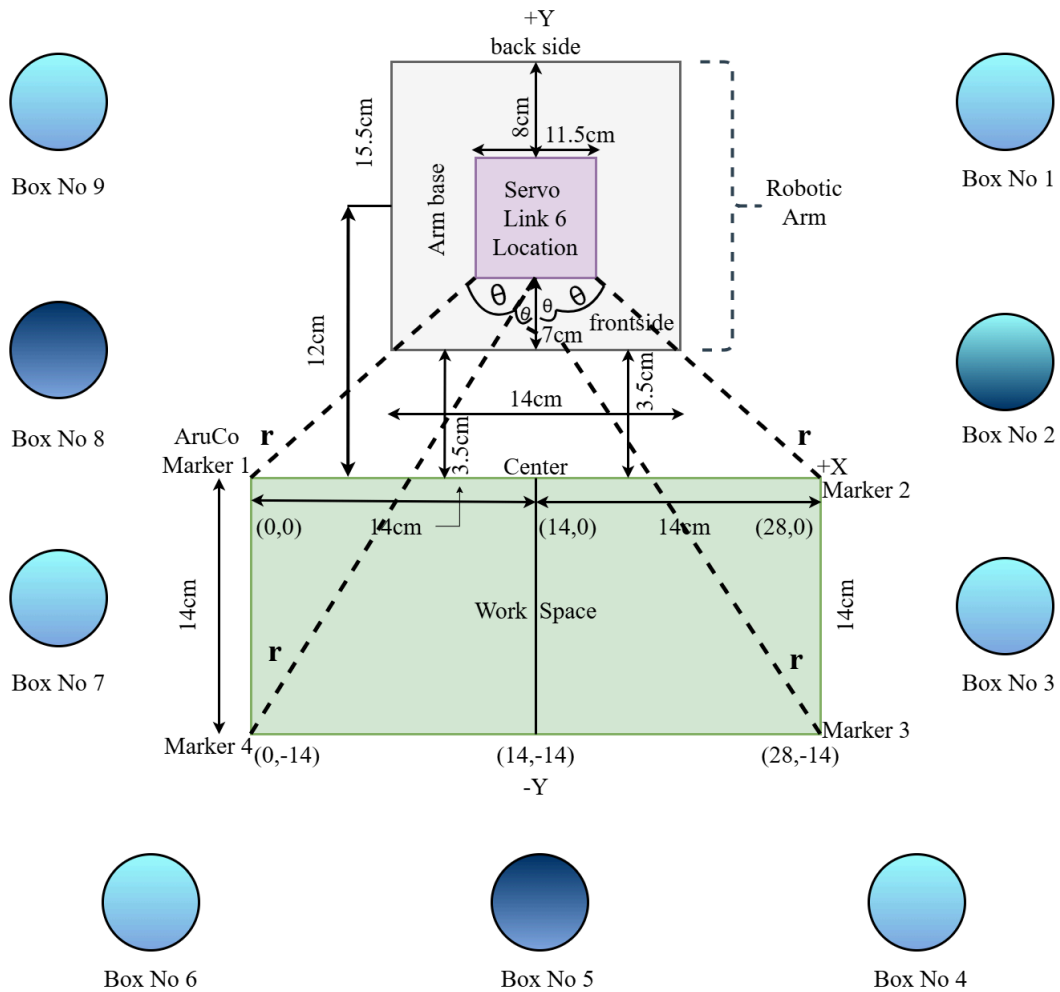


Figure 3.11: System Workspace

Let the detected image coordinates of the four markers be $P = \{(x_i, y_i)\}_{i=1}^4$. Due to potential variations in marker detection order, these points are first reordered into a canonical configuration shown in Equation 14:

$$P_{rect} = \{p_{tl}, p_{tr}, p_{br}, p_{bl}\} \quad (14)$$

corresponding to the upper-left, upper-right, lower-right, and lower-left corner points, respectively, as indicated in Figure 8. Such an arrangement ensures that there is a geometric relationship that must be maintained in order to ensure accurate homography estimation, avoiding any spatial uncertainty between frames. Following the rearrangement of the points, the transformed points can be assigned the corresponding workspace coordinates as (0,0), (28,0), (28,-14), (0,-14). A

Homography matrix $H \in \mathbb{R}^{3 \times 3}$ is computed to map image plane coordinates to the workspace plane in Equation 15:

$$\begin{bmatrix} w_x \\ w_y \\ 1 \end{bmatrix} = H \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

where (x, y) corresponds to pixel coordinates within the camera frame, while (wx,wy) indicates the physical coordinates within the workspace (in cm). The above equation compensates for 408 perspective distortion, creating a corrected top-down view of the workspace. This approach is based on the assumption that the workspace surface is flat, making the intrinsic camera parameters irrelevant in the calculation. After detecting the workspace boundaries, the transformation matrix H becomes constant, known as workspace locking, which is used for each frame without modification. This ensures consistency between frames regardless of any potential disturbances in the marker detection process. To optimize calculations in real-time operations, the rectified image undergoes normalization. With respect to an object's centroid (c_x, c_y) in the transformed image of dimensions (w, h), the physical coordinates are determined as shown in Equation 16:

$$w_x = \frac{c_x}{w} \times 28, \quad w_y = -\frac{c_y}{h} \times 14 \quad (16)$$

The negative sign in w_y accounts for the opposite direction of image and workspace coordinate systems. Here, it is assumed that the scale factor is consistent across both axes after the transformation. In order to make the camera coordinate system work in coordination with the robot coordinate system, the following transformations take place. In the workspace coordinate system, the location of the robot base (Servo Link 6) is at a constant height over the workspace with an offset defined in value 17.

$$(O_x, O_y) = (14, -12) \quad (17)$$

The conversion from the workspace coordinate system to the robot coordinate system is accomplished using the Equation 18.

$$r_x = w_x - O_x, \quad r_y = w_y - O_y \quad (18)$$

The target position with respect to the robot base is represented by (r_x, r_y) . For trajectory generation purposes, the polar representation of the target location is determined using Equation 19.

$$r = \sqrt{r_x^2 + r_y^2}, \quad \theta = \tan^{-1}\left(\frac{-r_x}{r_y}\right) \quad (19)$$

In the above equations, r refers to the radial distance between the robot base and target locations while θ refers to the base rotation angle. It is observed that with the proposed framework, the combination of ordered markers for calibration, homography, coordinate normalization, and the transformation with a known offset results in the efficient and robust conversion between the camera and robot coordinate frames.

3.2.4.3. Object Detection Framework

To achieve accurate detection and localization of vegetables in the calibrated workspace, the designed system utilizes an approach to object detection and tracking based on deep learning. Specifically, the object detection task utilizes a YOLO detector that was pretrained and fine-tuned according to the presented self-supervised learning (SSL) framework (as discussed earlier). In turn, a YOLO-based architecture has been chosen for its relatively high-speed inference and ability to conduct multi-class object detection in real time. The tracking mode is used to provide increased temporal consistency by maintaining object identity across frames.

For each input frame obtained from the rectified workspace, the model performs inference to generate bounding boxes, class labels, confidence scores, and unique tracking IDs. Let a detected object be represented in Equation 20.

$$D = \{(x_1, y_1, x_2, y_2), c, s, id\} \quad (20)$$

Where (x_1, y_1, x_2, y_2) are the coordinates of the bounding box, c is the class label, s is the confidence score, and id stands for the tracking identifier. In order to make the algorithm robust, a confidence threshold of τ_d is employed in Equation 21.

$$s \geq \tau_d \quad (21)$$

Where $\tau_d = 0.2$ is chosen as the filtering threshold for weakly confident detections. The centroids of the detected objects are computed by Equation 22.

$$Cx = \frac{x_1 + x_2}{2}, \quad Cy = \frac{y_1 + y_2}{2} \quad (22)$$

This becomes the key spatial feature that will be used later on. In order to minimize noise during localization and enhance temporal continuity, an exponential smoothing technique is employed in Equation 23.

$$\hat{c}_t = \alpha \hat{c}_{t-1} + (1 - \alpha)c_t \quad (23)$$

where $\alpha = 0.7$ is the smoothing parameter. Moreover, the stability filter approach is applied to guarantee accurate detections. An object is said to be detected only if it remains consistent in its existence for a minimum number of frames consecutively shown in Equation 24.

$$N_{\text{stable}} \geq 5 \quad (24)$$

Where N_{stable} is the number of frames an object needs to be detected persistently. This helps in eliminating false detections that can occur due to noise. The detection and tracking function is run all the time in real-time to update the state of objects detected within each frame. When the robot starts moving, detection results are frozen to avoid inconsistency brought about by disturbances. Through this process, which involves real-time detection, persistent tracking, filtering based on confidence score, centroid smoothing, and verification of consistency across frames, the proposed method offers an accurate perceptual system.

3.2.4.4. Tracking

In order to ensure the same object identity in adjacent frames, a tracker module, utilizing the ByteTrack approach, is included in the pipeline. Although the YOLO detector predicts object locations per frame, the tracker makes sure to keep objects' identity consistent through the use of temporal association techniques, making them appropriate for manipulation by robots. Define the set of detections at time t and the set of active tracks in Equation 25.

$$Dt = \{D_t^1, D_t^2, \dots, D_t^n\}, T_{t-1} = \{T_{t-1}^1, T_{t-1}^2, \dots, T_{t-1}^m\} \quad (25)$$

The goal of the tracker is to match each of the detections D_i to one of the current tracks T_{t-1}^j , based on their similarity. In this case, this can be done with the help of the IoU metric in Equation 26.

$$IoU(D, T) = \frac{|D \cap T|}{|D \cup T|} \quad (26)$$

ByteTrack uses a two-phase approach to associate detections to tracks. The first phase is to match high-confidence detections to tracks using an assignment with IoU. Low-confidence detections are then used to attempt any missed assignments for

increased robustness under partial occlusions and temporary poor detection conditions. The detection D_t^i associates with track T_{t-1}^j when the condition is satisfied through Equation 27 .

$$IoU(D_t^i, T_{t-1}^j) \geq \tau_{match} \quad (27)$$

τ_{match} refers to the matching threshold.

In terms of ByteTrack application, it is highly recommended because the algorithm is designed in a way that it can assign a unique stable identity to even low-confidence detections. Low-confidence detections are common in real applications due to partial occlusions, changes in lighting, and small movements of objects. This helps ensure that the system maintains a consistent association for each vegetable instance from detection until they leave the workspace. Persistent tracking will be helpful in monitoring object paths, object disappearances, and other higher processes, such as stability filtering and retry validation to avoid any external sensors.

3.2.4.5. Motion Planning and Servo Control

The module of motion planning and control is tasked with translating the target coordinate position relative to the robot into coordinated servo action necessary for manipulation. With the transformation performed to the target coordinate system (r_x, r_y) , the relationship between the position of the robot's base and the target is articulated through Equation 19.

This transformation corresponds to an inverse kinematics (IK) formulation, where the desired end-effector position in the Cartesian space (r_x, r_y) is mapped to the corresponding joint configurations of the robotic arm. In general, inverse kinematics involves determining joint variables that achieve a specified end-effector pose. In this work, instead of solving a full analytical or iterative IK model, a geometric approximation is adopted by decomposing the motion into radial distance and angular orientation. This allows direct computation of servo positions based on (r, θ) , enabling efficient real-time control while maintaining sufficient accuracy for pick-and-place operations. In this case, r is the radial component and θ the angular component of movement required to reach the target. Such a separation helps achieve a direct connection with the kinematics of the robotic arm. To ensure feasible operation, the radial distance is normalized within the reachable workspace in Equation 28.

$$r_n = \frac{r - r_{min}}{r_{max} - r_{min}}, \quad r_n \in [0, 1] \quad (28)$$

with r_{max} and r_{min} representing the minimum and maximum possible range of the robot arm, respectively. The control of the robot is accomplished by manipulating three main servos: the base rotation (Servo 6), the radial motion (Servo 5), and the vertical motion

(Servo 4). All servos move based on their respective mapping functions using the geometrical characteristics in Equation 29.

$$s_6 = f_6(\theta), \quad s_5 = f_5(r_n), \quad s_4 = f_4(r_n, \theta) \quad (29)$$

Servo Manipulation Parts:

1. Base Rotation (Servo 6):

The base servo dictates the orientation angle of the robot arm and is calculated based on the rotation angle (θ) as shown in Equation 30.

$$\begin{aligned} s_6 &= s_{6c} - \frac{\theta}{\theta_{max}}(s_{6c} - s_{6min}), \quad \theta \geq 0 \\ s_6 &= s_{6c} + \frac{|\theta|}{\theta_{max}}(s_{6max} - s_{6c}), \quad \theta < 0 \end{aligned} \quad (30)$$

2. Radial Extension (Servo 5):

The radial extension of the arm is governed by Servo 5, which varies according to the normalized radial distance in Equation 31.

$$s_5 = s_{5max} - r_n^\gamma (s_{5max} - s_{5min}) \quad (31)$$

where γ determines the scaling effect nonlinearly for better accuracy.

3. Vertical Positioning (Servo 4):

The vertical positioning of the arm is determined by Servo 4, which depends on a coupling between distance and angle as shown in Equation 32.

$$\begin{aligned} s_4 &= s_{4min} + h(r_n, \theta)(s_{4max} - s_{4min}) \\ h(r_n, \theta) &= (1 - r_n) + \lambda(1 - r_n)^2 \left(1 - \frac{|\theta|}{\theta_{max}}\right) \end{aligned} \quad (32)$$

Such a formula guarantees that the arm moves correctly and stably towards the object regardless of its orientation.

Constraint of Servo Actuation Coordination:

In order to move smoothly without collisions, there should be constraints on servo actuations combined in Equation 33.

$$s_i = \text{clip}(s_i, s_i^{min}, s_i^{max}), \quad i \in \{4, 5, 6\} \quad (33)$$

Moreover, coordinated work can be provided through synchronizing the servo actuation that guarantees correct trajectory tracking and prevents abrupt mechanical motions in Equation 34.

$$\frac{ds_4}{dt} \approx \frac{ds_5}{dt} \approx \frac{ds_6}{dt} \quad (34)$$

Finally, for precise picking, we should apply offsets in order to obtain desired positions during the pick phase in Equation 35.

$$s'_4 = s_4 + \Delta_4, \quad s'_5 = s_5 - \Delta_5 \quad (35)$$

The offset Δ_4 allows adjusting vertical displacement while Δ_5 changes forward reach. Thus, geometric motion planning in combination with coordinated servo actuation results in efficient and stable manipulation within the workspace.

3.2.4.6. Autonomous Execution Strategy

Control over system operations is provided by a finite state machine as presented in Figure 3.12, combining perception and robot execution steps. The process starts with the Detecting step, employing the detection and tracking approach based on YOLO. After that comes Selecting and Target Locking, where an appropriate object for manipulation is selected, and the detection process is locked. Next, the Picking step follows, during which the robotic manipulator executes picking. The result is validated by the Trajectory Validation stage, which assesses the success by detecting the object displacement and disappearing from the scene. In case of a positive validation result, further steps involve Placement and Post-validation steps, after which the Reset is executed. Otherwise, in case of failure in picking, the process reiterates up to a certain number of attempts. Otherwise, the system is aborted and reset.

– Finite State Machine Description:

The overall process is implemented using a finite state machine having two distinct states: DETECTING and ARM_BUSY. The DETECTING state includes tasks such as detection, tracking, and stability analysis. After identifying a proper object, the system switches to the ARM_BUSY state and starts performing the robotic function in Equation 36.

$$State_t = \begin{cases} DETECTING, & \text{if no tasks present} \\ ARM_BUSY, & \text{if pick is in progress} \end{cases} \quad (36)$$

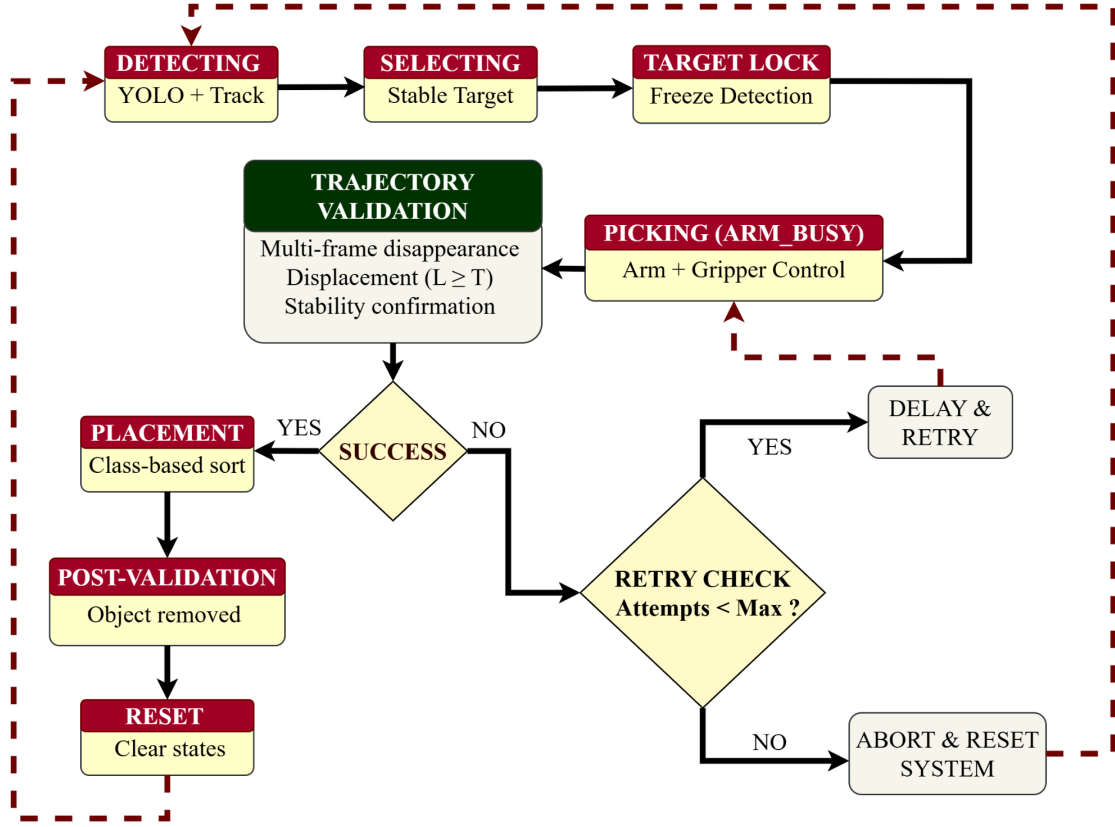


Figure 3.12: State Machine Diagram

In the ARM_BUSY state, a detection hold technique is used to stabilize the object to be processed. The number k , indicating a task index, should not overlap between different tasks shown in Equation 37.

$$k \leftarrow k + 1 \quad (37)$$

-Validation and Retry Strategy using Trajectories:

For the reliable functioning of the algorithm without the help of any external sensing mechanism, the approach of validating the trajectories generated along with detecting the presence of objects is used. The set of positions for every object under tracking and the distance between the first and last point of the trajectory is determined by Equation 38. Also, a trajectory validation consideration is shown in Equation 39.

$$P = \{p_1, p_2, \dots, p_n\}, \quad L = \|p_n - p_1\| \quad (38)$$

$$L \geq L_{min} \quad (39)$$

This will ensure that only significant object movements are taken into account, ignoring any insignificant noise introduced due to tracking errors.

For each pick trial, two criteria are evaluated:

1. **Disappearance criterion:** The object disappears from the workspace.
2. **Trajectory criterion:** The object shows valid movement before disappearing.

In the event that both criteria are met, a successful pick is achieved. In other cases, a retry mechanism is activated through the following inequality in Equation 40.

$$N_{retry} \leq N_{max} \quad (40)$$

Here, N_{max} is the upper limit for retries. In cases where the object is still detectable or does not satisfy the trajectory criterion, a delayed retry is launched. When the number of retries reaches the upper limit, the robot restarts the detection process.

-Classification-Based Placement Scheme:

The robot places the object in a target location based on its classification after a successful pick. This is done by applying a mapping function that maps each object class c to a placement basket like the given function 41.

$$B = f(c) \quad (41)$$

New vegetables are sorted among different placement bins, whereas damaged ones are sorted in a specific reject bin. Target locations are set according to offset distances between servos shown in Equation 42.

$$s_i = s_i^{home} + \Delta_i(B) \quad (42)$$

where $\Delta_i(B)$ represents the servo-specific offset for the selected bin. The robotic arm executes a structured sequence to reach the bin, release the object, and return to the home position, ensuring consistent and organized sorting.

Overall, the integration of state-based control, trajectory-aware validation, adaptive retry logic, and class-based placement enables a robust, sensor-free, and fully autonomous robotic manipulation system.

3.2.5. Realtime Web Application Dashboard

This paper also built a real-time web application, based on an **OpenCV-driven** dashboard, which is a web-based interactive and user-friendly interface for analyzing vegetables. An integrated live camera detection feature is supported to enable real-time viewing, which is best suited for continuous monitoring, experimental testing, or dynamic data capture. Annotated video clips and processed images can be saved, allowing users to store results for further examination. The dashboard also provides dynamic visualization of measurements with live plots, facilitating easier interpretation of results in real time and improving data-driven decision-making. In addition to detection and annotation functionalities, a dedicated image classification module is implemented, utilizing self-supervised learning (SSL) techniques. Moreover, the platform includes an object detection-based robotic arm control system used to sort vegetables

autonomously based on decisions. Based on the predicted class, the robotic arm can pick up and place vegetables at desired locations, enabling an end-to-end automation pipeline from detection to execution. Overall, the software seamlessly combines real-time detection, evaluation, visualization, and automated control, providing a comprehensive tool for both research and practical agricultural or food-processing applications.

3.3. Summary

This chapter presents the complete materials and methodology underlying an automated vegetable freshness detection and sorting system. The dataset was manually collected using five smartphones: Google Pixel 9, Samsung Galaxy S23, Vivo Y22s, Samsung Galaxy A55, and iPhone 15 Pro Max. Capturing images of eight non-seasonal vegetables: cucumber, tomato, bitter melon, potato, pointed gourd, onion, eggplant, and lemon. Each vegetable was photographed in both fresh and damaged conditions, creating 16 subclasses. Images were taken against white backgrounds in single-class isolated, single-class combinational, and multi-label configurations, where vegetables were paired as tomato-cucumber, bitter melon-pointed gourd, potato-onion, and eggplant-lemon. The final dataset comprises 4,640 images, balanced across all subclasses, annotated using the Roboflow platform with bounding box labels, resized to 640×640 pixels, and split into 70% training, 15% validation, and 15% testing subsets.

For detection, three supervised YOLO variants were employed, YOLOv10s, YOLOv11s, and YOLOv12s representing progressively advanced architectures, with YOLOv12s incorporating area-attention modules and FlashAttention for superior localization accuracy. These were benchmarked against three self-supervised learning methods: a DINOv2-style teacher-student framework using multi-crop augmentation and exponential moving average updates, a BYOL-style pipeline using online and target networks with cosine similarity loss and no negative pairs, and an MAE-style approach that masks 75% of image patches and trains the backbone to reconstruct missing content. In all SSL approaches, the pretrained backbone weights were transferred to YOLO detectors and fine-tuned on the labeled vegetable dataset. All training was conducted on a Kaggle NVIDIA Tesla T4 GPU using PyTorch with the Ultralytics YOLO framework, running for 50 epochs with AdamW optimization, a learning rate of approximately 0.002 for supervised models and 5e-4 for SSL models, and consistent augmentation strategies including mosaic, HSV jitter, and horizontal flipping.

To improve model interpretability, two explainability techniques were integrated Grad-CAM, which generates gradient-weighted heatmaps highlighting class-relevant regions such as discoloration and damage zones, and Eigen-CAM, which uses PCA on feature maps to produce more stable, gradient-free activation visualizations. Together,

these methods provide both gradient-based and feature-based perspectives on model decisions, making predictions more transparent and trustworthy.

On the hardware side, a Hiwonder xArm 1S robotic arm with six degrees of freedom, of which four were actively used, was mounted on a wooden base with a fixed top-down RGB camera. The workspace was calibrated using four ArUco fiducial markers placed at its corners, enabling homography-based coordinate transformation that maps detected image-plane coordinates into physical robot workspace coordinates. Detected object centroids were smoothed using exponential filtering and validated across consecutive frames for temporal consistency. A geometric inverse kinematics approximation then translated target positions into servo commands for three joints controlling base rotation, radial extension, and vertical positioning. The entire manipulation pipeline was governed by a finite state machine cycling through detection, target locking, picking, trajectory-based validation, retry logic on failure, and class-based placement into designated bins for fresh or damaged vegetables.

Finally, a Streamlit-based web application was developed to unify all system components into a single interactive dashboard. It supports image and video upload, live camera detection, SSL-based classification with Grad-CAM and LIME explainability tools, real-time metric visualization, and direct robotic arm control, offering a complete end-to-end platform suitable for both research purposes and practical agricultural or food-processing applications.

Chapter 4

Results and Discussion

4.1. Performance Evaluation

4.1.1. Evaluation of Supervised Model Performance

The evaluation of the supervised models is shown in Table 4.1 where the performance of YOLOv10s shows strong detection capability but always remains the least effective of the three variants. In this context, with a precision of 96.9% and recall of 94.2%, the model identifies most objects with a reasonably low number of false positives. The F1-score is 95.5%, showing a balanced harmony between precision and recall. Moreover, with a high value of mAP50 at 95.8%, its performance falls off more noticeably when moving to the more severe IoU thresholds, as presented by its mAP50-95 of 77.4. In contrast, YOLOv11s significantly improves nearly all the metrics where precision increases to 97.7%, and recall rises to 96.1%. Compared to YOLOv10s, this model reduces both false positives and false negatives more effectively. Besides, the F1-score

equals 96.9%, which suggests a strong balance between precision and recall, allowing it to outperform YOLOv10s in its general reliability. However, YOLOv11s managed to increase its mAP50 to 97.5%, reflecting its enhanced recognition capability. This model reached a more robust mAP50-95 of 81.6%, which indicates a more accurate bounding-box localization under more tight constraints. As a result, YOLOv11s shows an appealing improvement both in the coverage of broad detection and in the spatial precision, thus positioning it as a more optimized and stable model than YOLOv10s.

Table 4.1: Supervised Model Performance

Model	Precision	Recall	F1-Score	mAP50	mAP50-95
YOLOv10s	96.9%	94.2%	95.5%	95.8%	77.4%
YOLOv11s	97.7%	96.1%	96.9%	97.5%	81.6%
YOLOv12s	98.8%	97.9%	98.3%	99.2%	85.7%

Finally, YOLOv12s provides the best performance among all models that have been compared and its metrics have continuously outperformed previous versions. With a precision of 98.8% and a recall of 97.9%, the model is outstandingly accurate and complete in detecting objects. This is further established with its F1-score of 98.3%, showing outstanding synergy between finding the true positives and avoiding misclassifications. In addition, YOLOv12s has an excellent mAP50 of 99.2%, which further confirms that it has the best capability of correctly detecting objects at conventional IoU thresholds. Its mAP50-95 is 85.7%, reflecting the strongest fine-grained localization performance and surpassing the marks of YOLOv10s and YOLOv11s by a large margin. Therefore, continuous improvement in all metrics clearly indicates that YOLOv12s has the most mature optimization, the most stable convergence behavior, and the most precise spatial detection capability, hence becoming the most effective model in this series.

4.1.2. Evaluation of SSL Architecture Performance

After the evaluation of the supervised models, YOLOv12s is chosen as the model that performs the best and further becomes the backbone architecture for SSL. DINOv2 style and BYOL become the proposed methods for SSL. The comparative performance results of the two SSL methods with YOLOv12s detection model are given in Table 4.2.

Firstly, the BYOL initialized YOLOv12s shows high performance results with mAP50 and mAP50-95 equal to 98.4% and 83.1% respectively. Also, precision and recall are

97.6% and 96.8% respectively. Thus, it means that with BYOL, the model is able to detect most of the vegetables and show the correct classification of them. The BYOL method ensures good and reliable feature extraction without the need for a labeling process. However, the performance of DINOv2 style outperforms the performance of BYOL significantly. The DINOv2 style model reaches the mAP50 score of 99.6%, which is almost 1.2% higher in comparison with BYOL. Also, mAP50–95 increases to 88.3%. In addition, higher values of precision (99.1%), recall (98.5%) and F1-score (98.8%) confirm better performance of the DINOv2 method. Furthermore, the model built on MAE architecture also demonstrates good results, showing an mAP50 of 98.85% and mAP50–95 of 84.69%. Despite being a more improved version compared to the supervised baseline, it is still relatively less efficient compared to the DINOv2-based model since its feature representation is not as good. For further analysis, it should be noted that the DINOv2 model surpasses BYOL by 1.2% in terms of mAP50 and by 5.2% for mAP50–95 while showing superior results in comparison with the MAE model by 0.75% in terms of mAP50 and by 3.61%. In addition, precision and recall improvements over BYOL show even greater performance efficiency of the model.

Table 4.2: SSL Architecture Performance With Best Baseline Backbone

Architecture	Model	Precision	Recall	F1-Score	mAP50	mAP50 – 95
BYOL	YOLOv12s	97.6%	96.8%	97.2%	98.4%	83.1%
DINOv2-style	YOLOv12s	99.1%	98.5%	98.8%	99.6%	88.3%
MAE	YOLOv12s	98.12%	99.22%	98.76%	98.85%	84.69%

Finally, the improvements made possible through this model can be explained by its enhanced feature representation capabilities that capture the global context and keep feature invariance. In this way, it makes the detection task simpler and provides greater localization and classification capabilities of objects within images. All these SSL models provide improvement in the detection task; however, the one that performs the best is based on DINOv2.

4.2 Loss Curve and Clustering Evaluation

4.2.1 Loss Curve Performance of Supervised Models

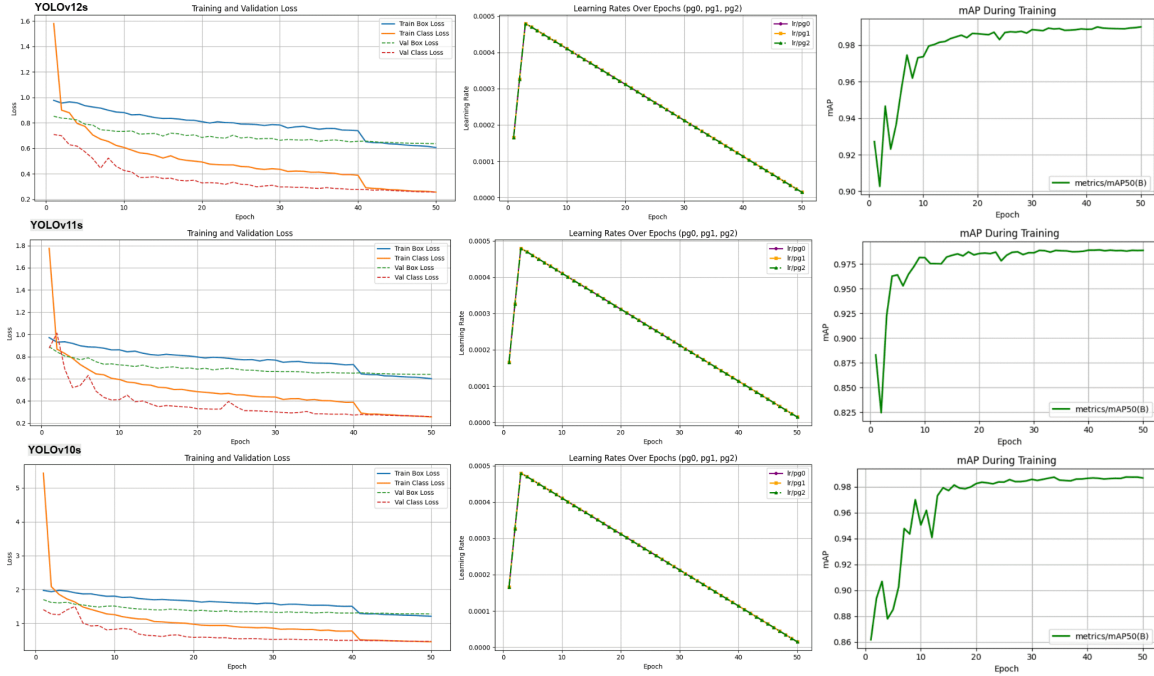


Figure 4.1: Loss, Learning and mAP Curve of Baseline YOLO Pretrains

After the evaluation of Supervised models, Figure 4.1 shows the loss, learning and mAP Curve of Baseline YOLO Pretrains. At First, YOLOv12s showed very stable convergence during training since box and class losses gradually decreased. Even though the losses on validation fluctuated, the general trend was to decrease, which is normal for object detection when objects in images have very different scales. Moreover, consistent optimization behavior was brought about by the nature of the learning rate schedule, it first warmed up and then linearly decayed. Accordingly, the mAP curve increased sharply at the beginning and reached above 0.98 after about epoch 20 then it stabilized. Overall, such results allow claiming that YOLOv12s manages to balance rapid learning, stable convergence, and high accuracy of detection, thus qualifying as a reliable candidate for real-time applications. Compared to YOLOv12s, the model YOLOv11s showed even smoother and more stable loss patterns, with more uniform decreases in both box and classification losses for the validation set. Besides, although the learning rate schedule was the same across the two models, YOLOv11s seemed to benefit from it more, therefore enjoying more efficient feature extraction in early epochs. Its mAP increased very fast, reaching over 0.97 in the very first several epochs and continuing to go up until reaching a plateau slightly higher than that of YOLOv12s. Therefore, YOLOv11s showed better generalization capability and slightly better localization precision. This means that YOLOv11s could be especially effective for datasets that require early-stage learning to play an essential role in the downstream detection performance. Unlike the first two models, YOLOv10s initially showed higher training

losses, especially in detection, which indicated a more difficult optimization phase within the first few epochs. However, as the training unfolded, the losses decreased steadily, and the validation curves showed steady improvements, which means it was adapting over time. Moreover, although this model made slower progress at the beginning, the learning rate strategy helped it gradually recover and stabilize. As a result, the mAP curve of the model rises more slowly but eventually approaches values close to 0.98, as for the other models. Hence, even though this YOLOv10s indeed needs more epochs to reach its best performance, it ultimately has very good detection accuracy. In some ways, that implies YOLOv10s suits applications in which longer training time is acceptable in exchange for competitive final results.

4.2.2 Loss and mAP Curve Performance of SSL Architectures

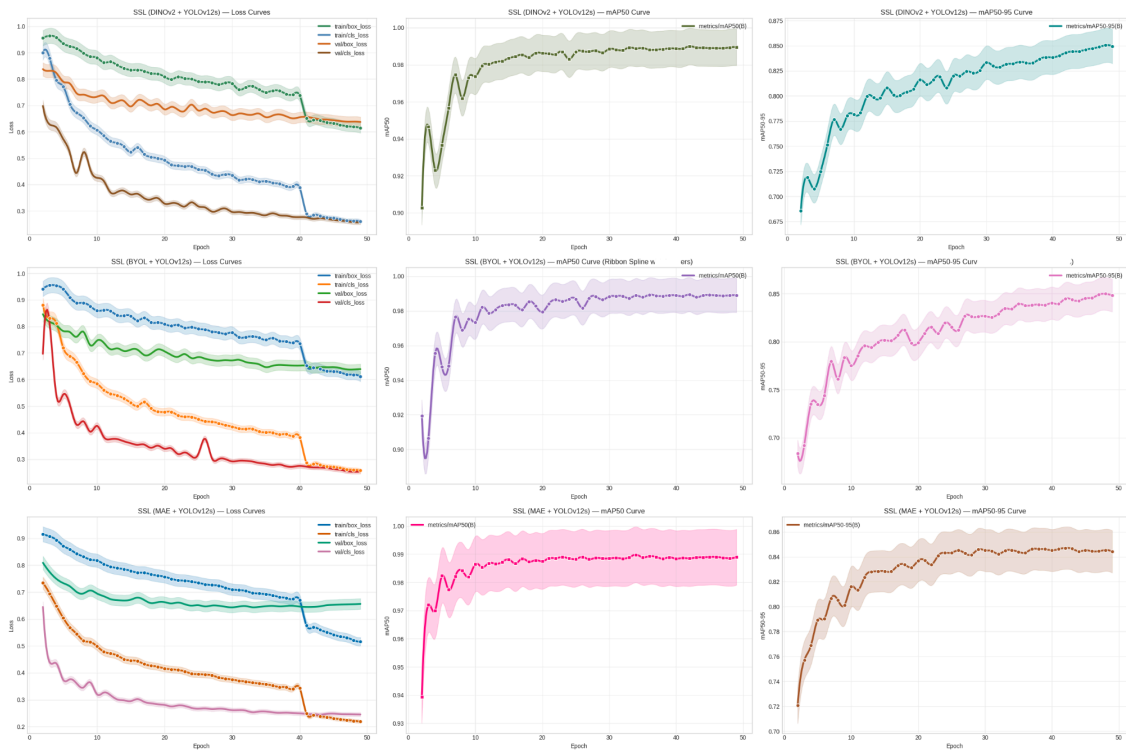


Figure 4.2: Loss and mAP Curve of SSL Architectures

In the same way as the supervised model curve plot, Figure 4.2 highlights the loss curve and the mAP curve under self-supervised learning techniques. As shown, the YOLOv12s trained using DINOv2 have excellent training properties where both the training and validation losses fall consistently as more epochs pass. Specifically, a significant drop is experienced following the warming-up stage, leading to convergence gradually. Similarly, there is a fast rise in the mAP50 curve up until its stabilization above 0.99 after exceeding 0.96. This implies that the YOLO model has high classification and localization capacity. Lastly, the mAP50-95 curve rises steadily to around 0.85 at the end of all epochs.

On the other hand, the SSL pipeline using BYOL also exhibits good results, albeit with

slightly unstable training dynamics. Despite the loss decreasing as time progresses, there are some changes, especially during the initial phases of training, which appear to be more pronounced than in DINOv2. The curve for mAP50 starts off high and continues in a manner similar to DINOv2 but has some oscillations before becoming stable at around 0.98. Likewise, the curve of mAP50–95 starts high and steadily grows but also displays some variability, eventually becoming close to 0.84.

Also, the SSL algorithm based on MAE presents smooth and stable converging results, where the loss graph gradually decreases and exhibits less fluctuation as compared to BYOL. The mAP50 graph increases sharply before stabilizing to about 0.99, suggesting excellent classifying capability. In the case of the mAP50-95 graph, there is a steady growth pattern that reaches about 0.845. Despite exhibiting higher stability than BYOL, the final localization result of the MAE algorithm still lags behind the DINOv2 method.

In the comparison of all the three SSL techniques, it can be seen that DINOv2 is more stable and robust, which is clearly indicated by its smooth loss curves and relatively higher mAP. Although MAE shows equally stable results, it does not perform well in terms of localization. BYOL proves to be highly efficient, although it presents greater variability in the training process than MAE. In conclusion, all three SSL models contribute greatly to the superior performance of YOLOv12s in contrast to the supervised model.

4.2.3 Clustering Analysis of SSL Architecture

Figure 4.3 illustrates the feature space learned by the SSL encoders using PCA and t-SNE visualizations. In the PCA projection, the clusters show considerable overlap, with fresh and damaged vegetables often positioned close together, making separation less clear. On the other hand, the t-SNE projection provides a sharper view, where categories such as cucumber and onion form tight clusters in both fresh and damaged states. In contrast, vegetables like lemon and potato appear more scattered, which may reflect their natural variability in texture and color. Overall, PCA captures only a portion of the variance, while t-SNE reveals more distinct boundaries, highlighting the importance of non-linear methods for analyzing SSL embeddings.

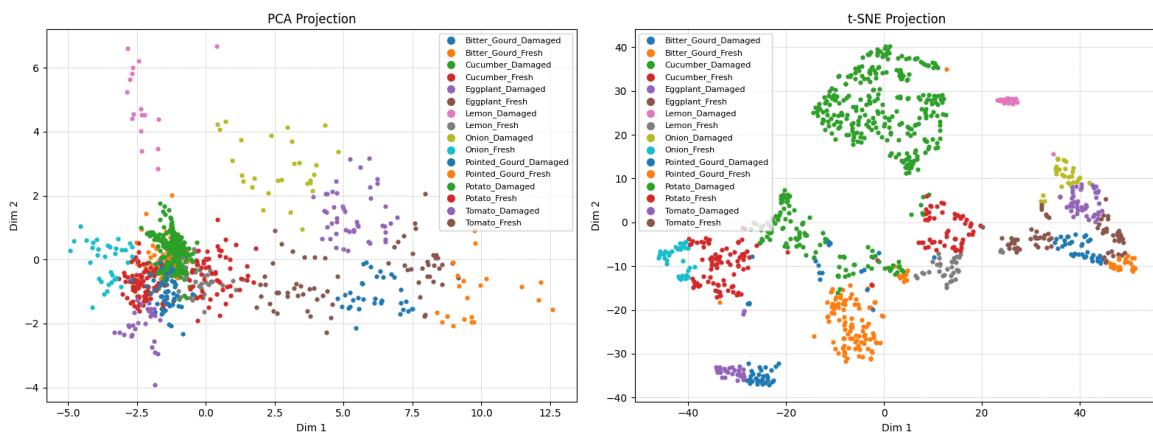


Figure 4.3: Clustering of DINOv2 Style Architecture using YOLOv12s

Figure 4.4 presents the PCA clustering results for the BYOL encoder, with an explained variance ratio of 21.8%. The plot shows moderate separation, with certain categories such as tomato and potato forming clearer distinctions between fresh and damaged samples. However, overlaps remain for visually similar vegetables, including cucumber and pointed gourd, suggesting that BYOL may struggle when spoilage features are subtle. This spread-out arrangement indicates that BYOL representations capture broader variability, which can be useful for handling diverse appearances but may reduce tight class-specific clustering.

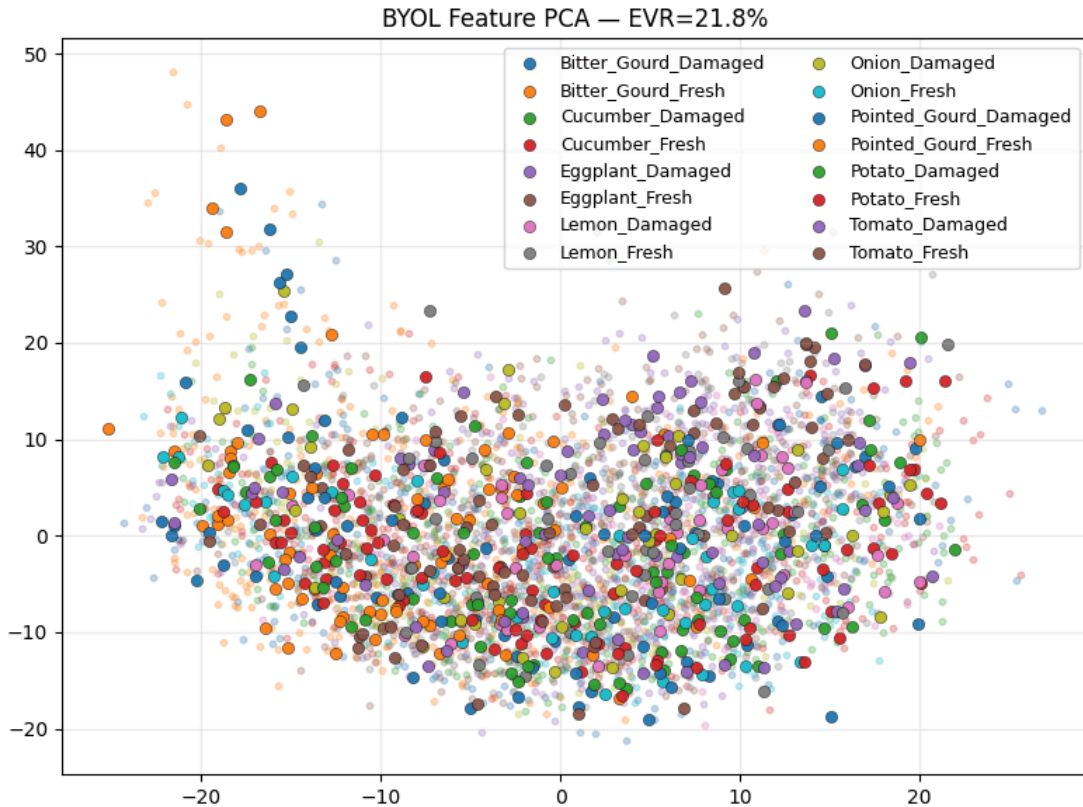


Figure 4.4: Clustering of BYOL Architecture using YOLOv12s model

In summary, the clustering analyses demonstrate complementary strengths of SSL models. DINOv2 produces tighter, more class-focused clusters, supporting accurate freshness classification, while BYOL generates a wider distribution of features that may improve generalization to new or varied examples. Together, these findings confirm that self-supervised learning can extract meaningful patterns without labels, and they help explain the detection performance observed in earlier experiments. This evidence also points toward future improvements in model design, where combining the strengths of both approaches could enhance agricultural freshness detection systems.

4.2.4 Confusion Matrix Analysis

Figure 4.5 presents the confusion matrix generated using the BYOL feature extractor with a kNN classifier on the validation set. The matrix shows strong diagonal dominance, confirming that most predictions align with true labels. For instance, Cucumber Fresh achieved 43 correct predictions, Onion Damaged 38, and Tomato Fresh 33, demonstrating the model's ability to capture visual cues such as texture, color, and shape. Misclassifications, however, highlight areas for refinement. Eggplant Fresh was often mistaken for Eggplant Damaged (16 cases), likely due to natural skin variations or lighting effects. Similarly, Tomato Fresh was misclassified as Tomato Damaged in 9 instances, reflecting the difficulty of detecting subtle spoilage cues. Cross-category confusions also occurred, such as Bitter Gourd Damaged vs. Pointed Gourd Damaged (7 cases each), and Potato Fresh misclassified as both Potato Damaged and Onion Fresh, suggesting overlapping visual traits. Overall, most errors occur within the same vegetable type rather than across categories, indicating reliable class distinction but weaker freshness discrimination. Classification accuracy exceeded 85% for most categories, consistent with earlier detection results. These findings confirm the effectiveness of BYOL-learned features while underscoring the need to improve sensitivity to subtle freshness differences and reduce ambiguity among visually similar damaged states.

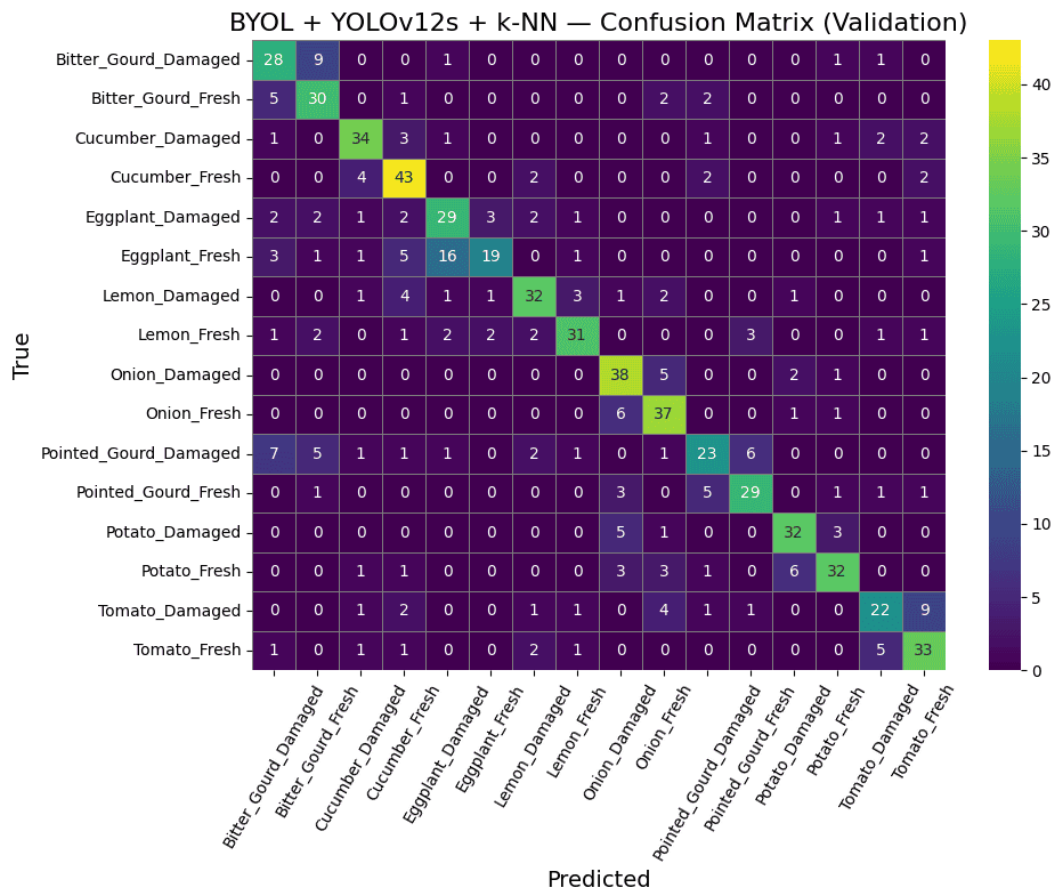


Figure 4.5: KNN Confusion Matrix after BYOL Feature Extraction

4.3 Detection Visualization of SSL Architectures

Qualitative analysis of the detection abilities of SSL-based frameworks is provided in Figure 16. This figure shows the localization and classification abilities of SSL-based architectures under various circumstances. The DINOv2-like framework depicted in the top part performs remarkably well for localizing vegetables since the predicted bounding boxes fit well and have very high confidence scores. They can detect defect areas and discriminate between healthy and unhealthy vegetables. On the other hand, the BYOL-like architecture represented in the middle part of Figure 4.6 successfully classifies the data but has slightly low localization abilities. It may occasionally produce incorrectly fitted bounding boxes when the environment is crowded. Similarly, the MAE-like architecture in the bottom part of Figure 4.6 provides consistent predictions with small errors.

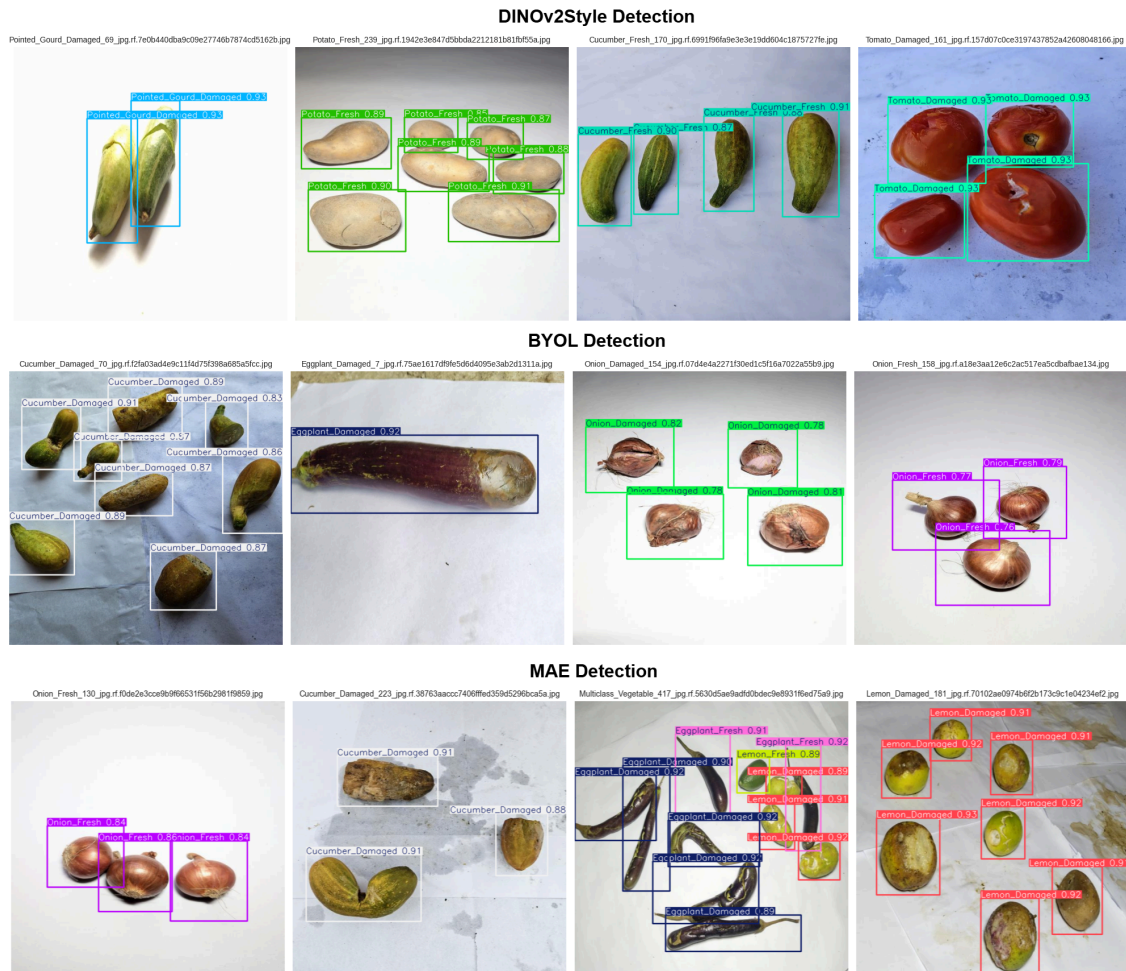


Figure 4.6: Vegetable Detection Visualized by SSL Architecture

In conclusion, all three SSL methods can identify vegetables accurately despite difficult circumstances, but DINOv2 performs better than both MAE and BYOL. The conclusions from this qualitative analysis agree with the quantitative data presented earlier, indicating

that pretraining using self-supervised learning is applicable to agriculture and other vision-related industries.

4.4 Explainable AI Techniques

Explainable AI (XAI) methods were used to interpret how the classification model distinguishes between fresh and damaged vegetables. These techniques help verify whether the model attends to relevant visual cues such as surface texture, color variation, and structural defects. Figure 4.7 shows an EigenCAM visualization applied to a sample input. The left panel displays the original image, while the right panel presents the corresponding heatmap. The model's attention is concentrated on blemished regions of the vegetable, avoiding the background and edges. This localized activation confirms that the model associates visible spoilage with the damaged label. Heatmap intensity reflects feature importance: brighter areas indicate stronger influence on the final prediction. In most cases, the model focuses on semantically meaningful zones, aligning with human judgment. However, occasional activation in background regions suggests sensitivity to noise, pointing to potential areas for refinement in preprocessing or augmentation. Overall, XAI visualizations enhance trust in the model's behavior and support its deployment in real-world sorting tasks. By revealing the basis of each decision, these tools ensure that predictions are both accurate and interpretable.

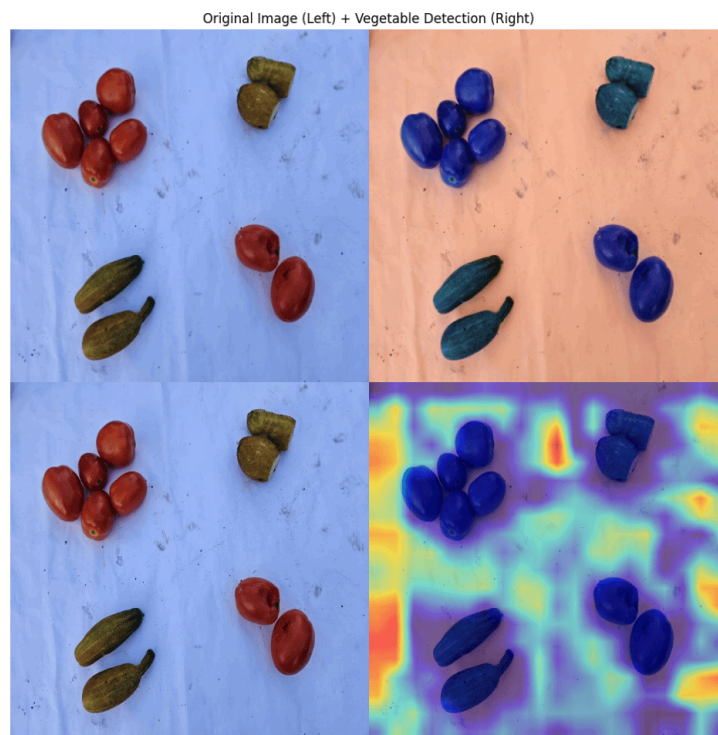


Figure 4.7: EigenCAM Visualization and HeatMap

4.5 Robotic arm Performance

The experimental platform that was built for implementing real-time object detection and manipulation by the robot is shown in Figure 4.8. A smartphone placed at a wooden stand is employed as a top-down camera to take real-time video of the workspace. The video frames are transferred to the host computer where an object detection and classification algorithm based on the YOLO framework is run to perform the inference process in a self-supervised learning (SSL) manner. Pick and place actions are performed by a robotic arm located at the center of a wooden base using object coordinates found in the workspace enclosed by four ArUco markers.

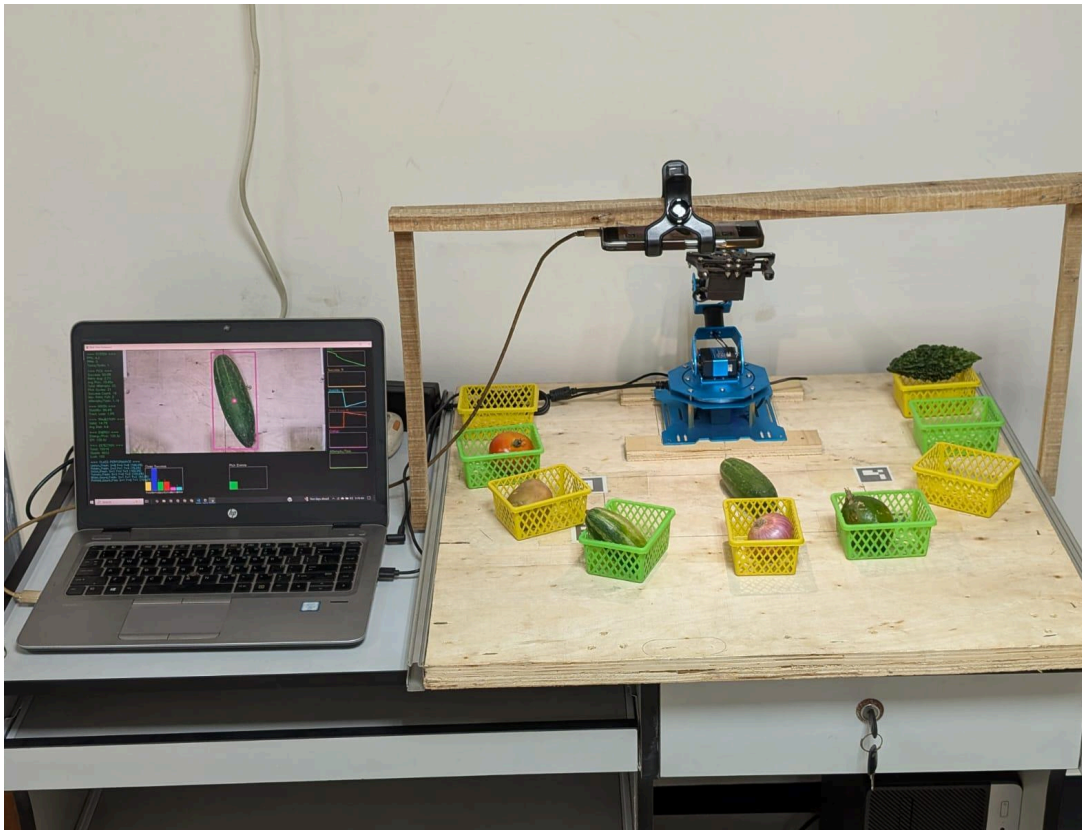


Figure 4.8: Experimental Platform

In addition, there is a real-time monitoring interface that displays bounding boxes and system logs for verification purposes. Figure 4.9 presents the step-by-step execution of the operation of the vision-guided robotic arm for object handling. Initially, the robot operates in the Idle state in which it stands still until the trigger for detection occurs through the vision module. Once an object is recognized, the arm proceeds to the Grab stage where it adjusts itself according to the object and starts gripping it. Next, during the Pick stage, the object is picked up and carried off in a fixed orientation to avoid any slipping. Finally, during the Place stage, the pick operation is confirmed through the tracking module by checking if the object has been moved from its original position. If confirmed, the object is placed at a specific location, otherwise, the system resorts to retrying the operation.

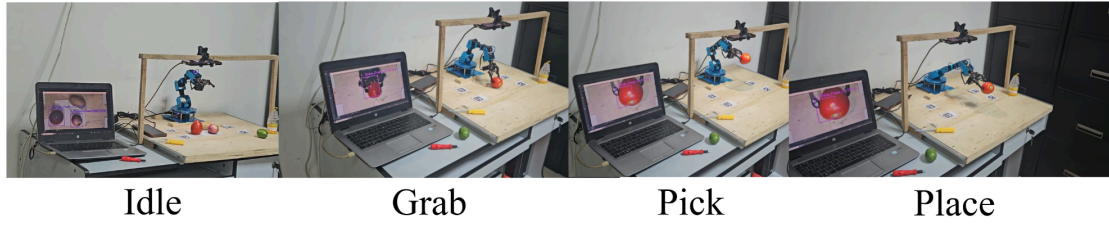


Figure 4.9: Sequential Operational Workflow

Table 4.3 shows the class-wise results obtained for pick and place during a time span of 5 minutes. On average, the success rate obtained was 89.6%, indicating the system's capability in grasping various types of vegetables with a high success rate. For instance, some classes that have been classified successfully include Cucumber_Fresh, Lemon_Fresh, Pointed_Gourd_Fresh, and Potato_Fresh, whose success rates were all 100%. Conversely, poor success rates are recorded for Tomato_Fresh and Bitter_Gourd_Fresh, with 71.4% and 80.0%, respectively. This might be because of the irregular shape of these vegetables and the smooth surfaces which cause difficulty in gripping them. Finally, a success rate of 85.7% is recorded for the Damaged category.

Table 4.3: Class-wise pick-and-place performance over a 5-minute evaluation period

Class	Success	Fail	Total	Success Rate (%)
Bitter_Gourd_Fresh	4	1	5	80.0
Cucumber_Fresh	5	0	5	100.0
Eggplant_Fresh	4	0	4	100.0
Lemon_Fresh	5	0	5	100.0
Onion_Fresh	4	1	5	80.0
Pointed_Gourd_Fresh	5	0	5	100.0
Potato_Fresh	5	0	5	100.0

Tomato_Fresh	5	2	7	71.4
Damaged	6	1	7	85.7
Total	43	5	48	89.6

Based on the class-wise evaluations above, Table 4.4 provides the summary of the overall performance metrics of the system. The robot is capable of executing 48 pick cycles in 5 minutes and has a total average of 9 pick tasks per minute. This result means that the system operates at an almost optimum capacity because its operating limit is set at this value. Apart from the above metric, the real-time capability of the system can be gauged by its average frame rate, which stands at 29 FPS. Further, from the data shown above, it is clear that on average, each pick cycle takes 3.10 seconds and has an average of 1.18 picks per task.

Table 4.4: Core system performance metrics computed from runtime execution

Metric	Value
Total Pick Cycles	48
Successful Picks	43
Success Rate (%)	89.6
Average FPS	29.0
Tasks per Minute (PPM)	9
Tasks per 5 Minutes	48
Average Pick Duration (s)	3.10
Average Attempts per Task	1.18

Retry Rate	1.18
Max Retry Failure Count	5

Along with the performance of the entire system, the efficiency of the perception module in terms of detection stability and tracking consistency can be assessed using the data provided in Table 4.5. In total, the system generates 320 detections, among which 280 can be considered stable; thus, the percentage of detection stability equals 87.5%. Moreover, the loss rate does not exceed 6.25%, which guarantees that the object identity will be preserved in the process of tracking using ByteTrack. Particularly, the disappearance-based success rate (89.6%) coincides with the picking accuracy rate (Table 11), confirming the validity of using object disappearance as an indicator of picking success. Thus, the perception module produces stable spatial data for further robotic operations.

Table 4.5: Detection and tracking performance metrics

Metric	Value
Total Detections	320
Stable Detections	280
Detection Stability Ratio (%)	87.5
Track Loss Count	20
Track Loss Rate (%)	6.25
Disappearance-Based Success Rate (%)	89.6

As an addition to the confirmation of the correctness of the performed picks, Table 4.6 provides trajectory-based validation results. The mean trajectory length is equal to 58.3 pixels, which is considerably above the set validation threshold of 15 pixels, suggesting obvious object relocation on successful picks. As a result, the percentage of correct trajectories reaches 91.2%. Another factor in favor of the reliability of the proposed

sensor-less validation technique is the correlation between trajectory and disappearance-based validation. In other words, it is possible to state that the trajectory analysis is able to determine whether the pick operation was successful.

Table 4.6: Trajectory-based validation metrics for pick success verification

Metric	Value
Average Trajectory Length (pixels)	58.3
Trajectory Validation Threshold (pixels)	15
Valid Trajectory Ratio (%)	91.2
Average Trajectory Length (mean)	58.3

Lastly, Table 4.7 summarizes the power consumption attributes of the robot. Through utilizing the real-time voltage values provided by the robotic controller, it is clear that the average operating voltage is 7.67V. From the average voltage value, the average energy consumption per pick achieved is estimated to be 35.7 J. The average energy consumption of 1711.9 J, derived from the picks performed during the experiment, gives a much better understanding of the operational power consumption of the robot. By taking into consideration both the real-time voltage values and also using the time taken for tasks as a measure of power consumption, a much better understanding of energy consumption can be achieved.

Table 4.7: Task-based energy consumption metrics for pick-and-place operations

Metric	Value
Measured Average Voltage (V)	7.67
Estimated Operating Current (A)	1.5

Average Pick Duration (s)	3.10
Total Pick-and-Place Tasks	48
Total Active Operation Time (s)	148.8
Average Energy per Pick (J)	35.7
Total Energy for Tasks (J)	1711.9

4.6 Real-Time Web Application Dashboard

Figure 4.10 shows the integrated web-based dashboard that was implemented for the operation and monitoring of the entire vegetable-sorting system. The interface integrates various functional modules together, which can support seamless interactions among vision-based detection, robotic actuation, and system-level configuration. The Automation Panel supplies one-click control for initiating and terminating end-to-end automated sorting, processing in real time from camera stream inputs down to object handling operations. The Live Detection Module allows both webcam and smartphone-based video feeds, with further support for uploading image and video files; it allows users to preview the results, download annotated outputs, and configure detection parameters such as thresholds in confidence and debugging overlays. The dashboard also includes a dedicated Plot Visualization Module that allows users to inspect statistics on detection, class distributions, and historical system performance directly from the interface. For fine-grained object-wise execution, the Class-wise Pick & Place Module reads class definitions from the systems data.yaml file to enable focused manipulation of specific object categories. Also, the dashboard shows the live graphs on the right and evaluation metrics on the left to continuously monitor the task. Altogether, this dashboard represents a centralized control environment that unifies perception, decision-making, visualization, and actuation into one coherent and operator-friendly system.



Figure 4.10: OpenCV-based Dashboard

4.7 Discussion

4.7.1 Summary of Findings

The results from the experiment clearly show that the proposed framework performs very well in detecting vegetables and classifying them based on freshness. Of all the supervised models, YOLOv12s shows the best stability and accuracy in performing its task, with a mAP50 score of 99.2% and excellent convergence and balance between precision and recall, as demonstrated in Table 3.7. Moreover, the model is able to locate the objects accurately, with a localization performance score of 85.7% in mAP50:95, which suggests high spatial consistency in recognizing fresh/damaged vegetables. On the other hand, YOLOv10s performs better in terms of efficiency but shows poor localization performance, with a mAP50:95 score of 77.4%.

From the analysis of training dynamics in Figure 3.13 above, we realize that the model YOLOv12s is able to consistently converge, with loss curves progressively reducing to mAP50 greater than 0.98 within around 20 epochs. The other models like YOLOv11s and YOLOv10s have smoother learning processes with YOLOv11s being able to generalize from an earlier stage compared to YOLOv10s whose convergence process is slightly delayed.

Furthermore, there have also been significant improvements made on how the features are represented through the use of self-supervised learning. The use of pre-training using a DINOv2 style, as well as BYOL and MAE models has shown to be effective in enhancing detection outcomes, allowing for the generation of a DINOv2 pre-trained YOLOv12s model with an mAP50 value of 99.6% and an mAP50:95 value of 88.3%, as shown in Table 8. This improvement is further backed up by better convergence and

stability during training, as illustrated in Figure 3.14. Furthermore, the clustering results validate the conclusions drawn above. In particular, DINOv2 generates compact clusters with high separation, whereas BYOL demonstrates wide variance yet relatively poor discriminatory power, as depicted in Figure 4.2. Moreover, as shown in Figure 4.4, it is found that most mistakes happen between visually identical types of vegetables, indicating the challenge of distinguishing fresh vegetables based on their subtle differences. However, overall, more than 85% classification accuracy can be achieved by our system for most classes.

Explainability analysis using EigenCAM, illustrated in Figure 4.7, indicates that the model concentrates on relevant areas such as defects in the surface, discolorations, and texture differences caused by the damaged state. This helps in achieving human-like visualizations and hence improves system interpretability and trust.

Finally, Table 4.1 and Table 4.2 show effective real-time processing and coordination among various object classes. Moreover, according to Table 4.3 and Table 4.4, the process achieves consistency in perception and validation tasks, which leads to an excellent success rate. Lastly, the proposed vision-based robot grasping framework shows robustness and efficiency through the application of reliable detection, tracking, and validation via trajectory optimization. Moreover, the inclusion of validation without sensors and task-oriented energy evaluation, as shown in Table 4.5, increases system feasibility. Despite some fluctuations in performance because of object-specific grasps, the whole process shows reliability and scalability.

To summarise these observations, Table 4.8 provides a consolidated analytical comparison of supervised and self-supervised outcomes across accuracy, robustness, label efficiency, stability, and runtime performance.

Table 4.8: Analytical Comparison of Supervised and Self-Supervised Learning Outcomes Across Key Performance Dimensions

Factor	Supervised Learning	Self-Supervised Learning
Detection Accuracy (mAP50)	High accuracy; YOLOv12s achieves 99.2%	Improved performance; DINOv2-YOLOv12s achieves 99.6%
Localization Precision (mAP50-95)	Moderate; YOLOv12s reaches 85.7%; degrades under occlusion	More stable; DINOv2 achieves 88.3% with smoother convergence
Robustness to Illumination Variation	Performs well under moderate lighting changes; YOLOv12s most stable	Strong invariance; SSL features effectively handle lighting variations
Robustness to Occlusion	Performance degrades under heavy occlusion; YOLOv12s performs best	Enhanced feature representation improves occlusion tolerance

Label Efficiency	Requires fully annotated data; performance depends on dataset size	Label-efficient; pretraining requires no annotations
Training Stability	Generally stable; YOLOv10s shows early fluctuations	Highly stable; DINOv2 ensures smoother and faster convergence
Freshness Discrimination	Effective for clear differences; struggles with subtle spoilage cues	Improved separability; better distinction between fresh and damaged samples
Feature Clustering Quality	Not explicitly analyzed	Produces compact and discriminative feature clusters (DINOv2); BYOL captures variability
Inference Speed	Real-time capable; YOLOv10s fastest, YOLOv12s moderate	Comparable inference speed; additional cost only during pretraining
Domain Adaptation Potential	Limited generalization without retraining	Strong transferability across different vegetable types

4.7.2 Comparison with Current Literature

The findings of this study align with and extend recent advances in deep learning for agricultural object detection and freshness classification. Prior work has demonstrated the effectiveness of YOLO architectures, annotation-efficient learning, and self-supervised pretraining for improving detection robustness under limited annotation scenarios.

Among supervised approaches, Li et al. [12] proposed D3 1029 -YOLOv10 for tomato detection in greenhouse environments, achieving 91.8% mAP@0.5 while reducing parameters by 54%. Our YOLOv12s baseline surpassed this with 99.2% mAP50, further enhanced to 99.6% under DINOv2-style pretraining, reflecting the advantages of later YOLO generations and SSL initialization for multi-class vegetable freshness detection. Similarly, Khanna et al. [11] demonstrated strong multi-class detection using an ensemble YOLOv8n model, achieving 0.82 mAP across 67 categories. Our framework extends this by integrating freshness-aware subcategories and self supervised pretraining, significantly improving both detection accuracy and feature separability without requiring additional annotations.

Self-supervised learning continues to show promise for representation learning in agricultural imagery. Wang et al. [32] achieved over 91% accuracy in crop classification using contrastive SSL with limited labeled samples. Our integration of DINOv2-style and BYOL pretraining further validates this: YOLOv12s initialized with DINOv2 achieved mAP50 of 99.6% with more compact and discriminative embeddings. Clustering

analyses confirmed stronger feature organization compared to supervised baselines, aligning with the representational advantages reported in prior SSL research.

A key distinction of this study is the emphasis on end-to-end deployability. While previous works focused primarily on model design and training performance, they did not integrate explainability modules, robotic manipulation, or a real-time application layer. Our Streamlit-based web interface bridges this gap by enabling live vegetable detection and classification while coupling perception outputs with autonomous pick-and-place execution via the Hiwonder xArm 1S. The addition of EigenCAM-based XAI tools further ensures that sorting decisions are transparent and interpretable, a dimension underexplored in comparable agricultural detection frameworks.

Table 4.9 summarizes the methodological and performance differences between recent literature and the proposed framework, highlighting how this work advances the state of vision-guided agricultural automation.

Table 4.9: Comparison of Proposed Framework with Related Literature on Vegetable Detection, Self-Supervised Learning, and Agricultural Robotics

Study	Crop / Task	Method	Key Findings	Comparison with Our Work
Li et al. [12]	Tomato detection	D ³ -YOLOv10 with DyFasterNet, DLKA attention	mAP@0.5 = 91.8%, reduced FLOPs, realtime efficiency	Our DINOv2-YOLOv12s achieved 99.6% mAP50 with superior freshness classification across 8 vegetable types.
Khanna et al. [11]	Fruit & vegetable detection	Ensemble YOLOv8n (FVDNet) with focal loss	0.82 mAP across 67 classes	Our framework extends multi-class detection to freshness subcategories with higher accuracy and SSL-enhanced features.
Wang et al. [32]	Crop classification	CC-SSL contrastive learning	Over 91% accuracy with limited labels	Our DINOv2 and BYOL pretraining surpassed this,

				achieving 99.6% mAP50 with stronger feature clustering in vegetable freshness tasks
Khoiruddin et al. [15]	Fruit & vegetable classification	MobileNetV2 supervised CNN	97.31% classification accuracy	Our detection-based SSL framework achieves comparable accuracy while additionally providing bounding box localization and robotic integration.
Bu et al. [13]	Agricultural object detection	FVRT-DETR with Mamba backbone	Strong performance under occlusion and complex backgrounds	Our YOLO-based SSL pipeline achieves competitive performance with lower computational overhead and integrated deployment pipeline.
Ci et al. [38]	Agricultural robotics	SSL-NBV for 3D plant reconstruction	Effective robotic planning without ground-truth labels	Our work extends SSLbased robotics to real-time pick-and-place vegetable sorting with an end-to-end perception-to-action pipeline.
This Work	Vegetable freshness sorting	Supervised (YOLOv10–12s), Self-SL (BYOL,	DINOv2-YOLOv12s achieves 99.6%	First unified pipeline combining

		DINOv2) + XAI + Robotic Arm + WebApp	mAP50 and 88.3% mAP50-95; SSL boosts feature clustering; realtime robotic sorting validated	supervised and self-supervised detection with explainability, robotic manipulation, and interactive deployment for vegetable freshness sorting.
--	--	---	--	--

4.8 Summary

This chapter presents the results and discussion of the proposed vegetable freshness detection and sorting system, covering supervised model evaluation, self-supervised learning performance, explainability analysis, robotic arm execution, and a comparative review against existing literature.

Among the three supervised YOLO models evaluated, YOLOv12s consistently outperformed the others across all metrics, achieving a precision of 98.8%, recall of 97.9%, F1-score of 98.3%, mAP50 of 99.2%, and mAP50-95 of 85.7%, establishing itself as the most accurate and spatially precise model in the series. YOLOv11s offered a strong middle ground with smoother convergence and better early generalization, while YOLOv10s, though slightly slower to converge, remained competitive and suited to latency sensitive deployments. Based on these results, YOLOv12s was selected as the backbone for all self-supervised learning experiments.

Among the three SSL methods evaluated DINOv2-style, BYOL, and MAE. The DINOv2-style framework demonstrated the strongest performance, achieving a mAP50 of 99.6% and mAP50-95 of 88.3% when integrated with YOLOv12s, surpassing both BYOL and MAE by meaningful margins. BYOL delivered reliable results with slightly more training variability, while MAE exhibited stable convergence but lagged in localization precision. Loss curve analysis confirmed that DINOv2 produced the smoothest and most stable training dynamics, while clustering visualizations using PCA and t-SNE showed that DINOv2 generated tighter, more class-discriminative feature clusters compared to the broader but less precise distributions produced by BYOL. Confusion matrix analysis further revealed that most misclassifications occurred within the same vegetable category, particularly between fresh and damaged states of visually similar vegetables such as eggplant and tomato, indicating that the primary challenge lies in subtle freshness discrimination rather than inter-class confusion.

Explainability analysis using EigenCAM confirmed that the model correctly focused its attention on semantically meaningful regions such as surface blemishes, discolorations, and textural defects associated with damaged vegetables, with only occasional sensitivity to background noise, overall supporting the trustworthiness and interpretability of the model's decisions.

The robotic arm system achieved an overall pick-and-place success rate of 89.6% across 48 cycles performed within five minutes, operating at an average frame rate of 29 FPS and completing each pick in approximately 3.10 seconds. Classes such as cucumber, lemon, pointed gourd, and potato achieved 100% success rates, while tomato and bitter gourd showed lower rates due to their irregular shapes and smooth surfaces. Detection stability reached 87.5% with a track loss rate of only 6.25%, and trajectory-based validation confirmed a valid trajectory ratio of 91.2%, well above the 15-pixel threshold, validating the sensor-free pick success verification strategy. Energy consumption analysis estimated an average of 35.7 joules per pick and a total operational energy of 1711.9 joules across all tasks.

The Streamlit-based web dashboard successfully unified all system components, offering live detection, manual and automated pick-and-place control, servo tuning, detection visualization, plot analytics, and SSL-based classification with Grad-CAM and LIME explainability tools in a single operator-friendly interface.

Compared to existing literature, the proposed framework surpassed prior work such as D3-YOLOv10 for tomato detection, ensemble YOLOv8n for multi-class fruit and vegetable detection, and contrastive SSL approaches for crop classification, while additionally offering a fully integrated pipeline that combines supervised and self-supervised detection, explainability, real-time robotic sorting, and an interactive deployment dashboard a combination not previously achieved in comparable agricultural vision systems.

5.1 Overall Contributions

This work presented the development of a vision-guided robotic system for automated vegetable sorting that effectively integrated self-supervised learning with low-cost robotic manipulation. By using the DINOv2-style and BYOL-based architectures, it achieved strong detection performance and showed reliable differentiation between fresh and damaged produce across various conditions of imaging. Confusion matrix analyses demonstrated that class recognition was highly robust, and explainable AI visualizations verified that models indeed attend to semantically relevant regions, therefore supporting interpretability and trust. The effective integration of the robotic arm enabled the successful performance of pick-and-place operations with high consistency, hence validating the system's capability for real-time and autonomous manipulation tasks. Overall, the findings have shown the practicality and efficiency of SSL-based approaches in reducing dependency on large labeled datasets while sustaining high accuracy. This provides a scalable and sustainable pathway toward automated food processing. Despite these promising results, some limitations still remain that are worth further investigation. Freshness discrimination among visually similar vegetables, such as bitter melon and pointed melon, was challenging and required more refined SSL architectures, targeted ablation studies, and improved hyperparameter tuning. More importantly, this would require improving the diversity of the dataset using advanced augmentation techniques, including synthetic lighting variations, occlusion simulation, and texture-preserving transformations, since robust generalization in real-world agricultural environments depends on that. From a robotics perspective, further optimization of the end-effector design along with motion-planning algorithms will help reduce damage and facilitate irregular object geometries. Furthermore, real-time monitoring, interactive explainable AI, and modular control interfaces need to be integrated into an operational dashboard in order to extend system usability and deployment readiness. All of these will go a long way in further strengthening the robustness of the system and accelerating its transition toward large-scale commercial adoption in automated sorting pipelines.

5.2 Limitations and Future Works

The thing about this study is that it did well in different situations but there are still some limitations. First of all the data we used was collected in a controlled environment with a white background and special lighting. So we do not really know if this framework will

work in life situations like on a conveyor belt with different lighting or in a messy place. The framework might not work well in these situations because it was not tested in them. We need to do work to see if this framework will work in different places and situations. We want to test it in different environments to see if it can really be used in many situations. This will help us understand if the framework can be used in situations without any problems.

A second limitation concerns fine-grained freshness discrimination. The models we made did well when we looked at how well they could find things, but they did not do as well when we made the rules for what is a good match a lot stricter. This was especially true for things that look a lot like bitter melon and pointed melon. When we looked closer at what was going wrong we saw that the system was getting mixed up between things that are the same but just a little bit different, like an eggplant that is fresh and one that is damaged. This is a problem because it is hard to see the things that make something go bad like a small change in color or texture. We are going to keep working on this and try some things like using special tools to help the system see these little differences and make the system better at looking at the details of what it is seeing. The vegetable freshness is still something that we need to work on, and it is not easy to get right. We will try to make the system better at seeing the differences in vegetable freshness.

Another limitation pertains to the dataset scale and diversity. We have 4,640 pictures of vegetables that are divided into 16 groups. We made sure each group had about the same number of pictures. Each group of vegetables does not have a lot of pictures and that means we might not have all the ways that vegetables can look when they are spoiled. Vegetable pictures will be added to in the future by using computer tricks to make pictures like changing the light or adding things that block our view and by using pictures from different places and times of year. We will keep adding vegetable pictures to make our collection bigger and better.

Finally, our Streamlit app works well for real-time detection and robot control. We still need to make it better for multiple users and high-speed sorting, like in commercial lines. We plan to work on making the model smaller, more efficient, and able to run on cloud and edge devices. This will help us make it more scalable, use energy, and get it ready for use in farms and other industrial settings with the Streamlit app. We want to make it ready for use in industrial agricultural environments.

In summary, future research will focus on expanding cross-condition robustness, enhancing freshness discrimination at fine granularity, improving dataset diversity and scale, optimizing robotic manipulation for irregular product geometries, and advancing deployment efficiency on edge platforms. Addressing these limitations will broaden the applicability of the proposed framework across diverse agricultural processing contexts and operational requirements.

Bibliography

- [1] N. Gul, K. Muzaffar, S. Z. A. Shah, A. Assad, H. A. Makroo, B. Dar, et al., Deep learning hyperspectral imaging: a rapid and reliable alternative to conventional techniques in the testing of food quality and safety, *Quality Assurance and Safety of Crops & Foods* 16 (1) (2024) 78–97.
- [2] W. J. Feng, Y. L. Zhao, Y. Xie, L. Wei, Enhanced yolov5s-based apple detection for har1129 vesting robots using a multi-rotated box algorithm, *Informatica* 49 (18) (2025).
- [3] Y. Liu, C. Wei, S.-C. Yoon, X. Ni, W. Wang, Y. Liu, D. Wang, X. Wang, X. Guo, Devel opment of multimodal fusion technology for tomato maturity assessment, *Sensors* 24 (8) (2024) 2467.
- [4] R. Zhao, Y. Zhu, Y. Li, Cla: A self-supervised contrastive learning method for leaf disease identification with domain adaptation, *Computers and Electronics in Agriculture* 211 (2023) 107967.
- [5] N. K. Uppalapati, B.Walt, A. J. Havens, A. Mahdian, G. Chowdhary, G. Krishnan, A berry picking robot with a hybrid soft-rigid arm: Design and task space control., in: *Robotics: Science and Systems*, Vol. 95, 2020.
- [6] A. Khan, T. Hassan, M. Shafay, I. Fahmy, N. Werghi, S. Mudigansalage, I. Hussain, Tomato maturity recognition with convolutional transformers, *Scientific reports* 13 (1) (2023) 22885.
- [7] L. Zhang, J. Liu, Y. Wei, D. An, X. Ning, Self-supervised learning-based multi-source spectral fusion for fruit quality evaluation: A case study in mango fruit ripeness prediction, *Information Fusion* 117 (2025) 102814.
- [8] C.Wang, X. Yao, F. Ding, Z. Yu, A trajectory planning method for a casting sorting robotic arm based on a nature-inspired genghis khan shark optimized algorithm, *Mathematical Bio sciences and Engineering* 21 (2) (2024) 3364–3390.
- [9] J. Kim, G. Kim, R. Yoshitoshi, K. Tokuda, Real-time object detection for edge computing based agricultural automation: A case study comparing the yolox and yolov12 architectures and their performance in potato harvesting systems, *Sensors* 25 (15) (2025) 4586.
- [10] F. Ali,W. Tariq, A. Razzaq, A. Rehman, S. Sarfraz, N. Rajput, S. Ali, K. Fatima, S. Jameel, N. Liaqat, et al., From sensors to insights: The fusion of ai edge

computing and precision agriculture, *J. Agric. Biol.* 3 (2025) 160–170.

- [11] S. Khanna, C. Chattopadhyay, S. Kundu, Enhancing fruit and vegetable detection in unconstrained environment with a novel dataset, *Scientia Horticulturae* 338 (2024) 113580.
- [12] A. Li, C. Wang, T. Ji, Q. Wang, T. Zhang, D3-yolov10: Improved yolov10-based lightweight tomato detection algorithm under facility scenario, *Agriculture* (2024). doi:10.3390/agriculture14122268.
- [13] X. Bu, Y. Wu, H. Lv, Y. Yu, Fast and accurate detection of forty types of fruits and vegetables: Dataset and method, *Agriculture* 15 (7) (2025) 760.
- [14] W. Wang, A. Zhu, H. Wei, L. Yu, A novel method for vegetable and fruit classification based on using diffusion maps and machine learning, *Current Research in Food Science* 8 (2024) 100737.
- [15] M. Khoiruddin, S. Tena, Fruit and vegetable classification using convolutional neural network with mobilenetv2, *Journal of Applied Research In Computer Science and Information Systems* 2 (2) (2024) 203–210.
- [16] J. Steinbrener, V. Dimitrievska, F. Pittino, F. Starmans, R. Waldner, J. Holzbauer, T. Arnold, Learning metric volume estimation of fruits and vegetables from short monocular video sequences, *Heliyon* 9 (4) (2023).
- [17] H. Li, F. Shi, A detr-like detector-based semi-supervised object detection method for brassica chinensis growth monitoring, *Computers and Electronics in Agriculture* 219 (2024) 108788.
- [18] A. Casado-García, J. Heras, A. Milella, R. Marani, Semi-supervised deep learning and low-cost cameras for the semantic segmentation of natural images in viticulture, *Precision Agriculture* 23 (6) (2022) 2001–2026.
- [19] R. Johanson, C. Wilms, O. Johannsen, S. Frintrop, S3ad: Semi-supervised small apple detection in orchard environments, in: *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2024, pp. 7076–7085.
- [20] J. Zhou, J. Zhang, J. Chu, H. Huang, Y. Sun, Memoryteacher: Memory fusion and carl for semi-supervised pest region detection, in: *2024 7th International Conference on Advanced Algorithms and Control Engineering (ICAACE)*, IEEE, 2024, pp. 1505–1508.
- [21] X. Xu, D. Zou, J. Zou, S. Xia, W. Sheng, Mulmatch-cl: A semi-supervised teacher-student framework for robust crop segmentation in uav imagery, *IEEE*

Access (2025).

- [22] K. Bogomasov, S. Conrad, Efficient iterative semi-supervised object detection training for counting of fruits and vegetables, in: Proceedings of the 2023 7th International Conference on Advances in Artificial Intelligence, 2023, pp. 133–137.
- [23] B. Deng, Y. Lu, Semi-supervised weed detection in vegetable fields: In-domain and cross-domain experiments, ArXiv abs/2502.17673 (2025). doi:10.48550/arxiv.2502.17673.
- [24] A. Saleh, S. Hatano, M. R. Azghadi, Weed detection in challenging field conditions: A semi-supervised framework for overcoming shadow bias and data scarcity, arXiv preprint arXiv:2508.19511 (2025).
- [25] J. Li, D. Chen, X. Yin, Z. Li, Performance evaluation of semi-supervised learning frame works for multi-class weed detection, Frontiers in Plant Science 15 (2024). doi:10.3389/fpls.2024.1396568.
- [26] Z. Chen, F. Liu, X. Zhu, Y. Qi, M. Ghavamzadeh, Preference optimization via contrastive divergence: Your reward model is secretly an nll estimator, arXiv preprint arXiv:2502.04567 (2025).
- [27] R. Li, X. Wang, Y. Cui, Y. Xu, Y. Zhou, X. Tang, C. Jiang, Y. Song, H. Dong, S. Yan, A semi-supervised diffusion-based framework for weed detection in precision agricultural scenarios using a generative attention mechanism., Agriculture; Basel 15 (4) (2025).
- [28] K. Najafian, A. Ghanbari, I. Stavness, L. Jin, G. H. Shirdel, F. Maleki, A semi-self supervised learning approach for wheat head detection using extremely small number of labeled samples, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2021, pp. 1342–1351.
- [29] A. Ter-Sarkisov, R. Ross, J. Kelleher, Bootstrapping labelled dataset construction for cow tracking and behavior analysis, in: 2017 14th Conference on Computer and Robot Vision (CRV), IEEE, 2017, pp. 277–284.
- [30] L. Gomez-Zamanillo, A. Bereciartua-Perez, A. Elola, A. Picon, A. Alvarez-Gila, I. Egusquiza, R. Navarra-Mestre, Semi-supervised approach for automatic counting of whiteflies with small annotated dataset, IEEE Access 13 (2025) 55586–55598. doi:10.1109/ACCESS.2025.3552195.
- [31] J. Yang, Z. Wang, D. Chen, S. Xiao, A. T. Azar, A joint network of super-resolution

and active learning for agricultural land classification, *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing* (2024).

- [32] H. Wang, W. Chang, Y. Yao, D. Liu, Y. Zhao, S. Li, Z. Liu, X. Zhang, Cc-ssl: A self supervised learning framework for crop classification with few labeled samples, *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing* 15 (2022) 8704–8718.
- [33] M. R. Gupta, A. Sobti, Time2agri: Temporal pretext tasks for agricultural monitoring, *arXivpreprint arXiv:2507.04366* (2025).
- [34] T. Choi, O. Would, A. Salazar-Gomez, X. Liu, G. Cielniak, Channel randomisation: Self-supervised representation learning for reliable visual anomaly detection in speciality crops, *Computers and Electronics in Agriculture* 226 (2024) 109416.
- [35] Y. Liu, S. Zhou, H. Wu, W. Han, C. Li, H. Chen, Joint optimization of autoencoder and self-supervised classifier: Anomaly detection of strawberries using hyperspectral imaging, *Computers and Electronics in Agriculture* 198 (2022) 107007.
- [36] A. Y. H. Chai, S. H. Lee, F. S. Tay, P. Bonnet, A. Joly, Beyond supervision: Harnessing self-supervised learning in unseen plant disease recognition, *Neurocomputing* 610 (2024) 128608.
- [37] A. Al Mamun, D. Ahmedt-Aristizabal, M. Zhang, M. I. Hossen, Z. Hayder, M. Awrangjeb, Plant disease detection using self-supervised learning: A systematic review, *IEEE Access* (2024).
- [38] J. Ci, E. J. van Henten, X. Wang, A. K. Burusa, G. Kootstra, Ssl-nbv: A self-supervised learning-based next-best-view algorithm for efficient 3d plant reconstruction by a robot, *Computers and Electronics in Agriculture* 233 (2025) 110121.
- [39] J. Ci, E. J. van Henten, X. Wang, G. Kootstra, Robotic self-supervised local view planning for efficient and scalable 3d plant reconstruction across varying plant sizes, *Computers and Electronics in Agriculture* 238 (2025) 110839.

Appendix A

Mapping of Course and Program Outcomes

CSE400-A

CO	CO Descriptions	K	EP	K	EP
CO1	Integrate new and previously acquired knowledge to identify a complex real-life engineering problem related to automated vegetable sorting	(i) Background [K1, K2, K3] Vegetable sorting in agriculture is often done manually, which is slow and inconsistent [K1]. Issues like freshness detection, lighting variation, and object overlap make automation difficult [K2]. We studied computer vision techniques, object detection models, and robotic systems to understand this problem [K3].	(i) Background [EP1] Vision-based detection and robotic automation were explored as the foundation for solving real-world food sorting problems. This is the foundation of the engineering problem and solution design [EP1]. ((ii) Research Questions/Problem Statements [EP6] When it comes to solving a complex engineering problem, our focus lies in the challenges to automate robotic arm using SSL models to detect and sort vegetables. The key research questions	(i) Background: K1,K2 and K3 are checked	(i) Background: EP1 is checked.

			include: • How to sort vegetables fully automatically? • Which YOLO model performs best? • Can self-supervised learning improve results? • How to integrate vision with robotics? • How to ensure real-time performance? These questions guided our development of the SSL model, which directly aligns with EP6.		
CO2	Examine previous work, define the gap, and formulate the objectives for deep learning-based automated vegetable sorting	(i) Related Works [K8] Reviewed YOLO models, self-supervised learning (DINO, BYOL, MAE), and robotic systems. Identified issues in lighting and real-time	(i) Related Works [EP1] automated vegetable sorting challenges and previous approaches were evaluated. Technical gaps were identified, motivating a new SSL-based automated solution [EP1]. ((ii) Objectives [EP2,	(i) Related Works: K8 is checked	(i) Related Works: EP1 is checked. (ii) Objectives: EP2, EP6 and EP7 are checked.

		performance. [K8].	EP6, EP7] The objective was to develop a dataset and train YOLO-based models to identify the most effective one. This selected model was then used to evaluate SSL models, and the best-performing SSL model was integrated with a robotic arm. The system aims to achieve higher accuracy, minimize manual intervention, and enhance reliability [EP2, EP6]. It also addresses the lack of affordable automated solutions in developing countries [EP7]. (ii) Planned Methodology [EP2, EP6] SSL models were proposed. Integration with XAI tools like Eigen-CAM, and implementation via robotic arm were planned for automated		(ii)Planned Methodology: EP2 and EP6 are checked.
--	--	-----------------------	--	--	--

			vegetable sorting, and developing dashboard. Expected results aimed to match or exceed expert sorting and speed up detection [EP2, EP6].		
--	--	--	--	--	--

CSE400-B

CO	CO Descriptions	K	EP
CO3	Analyze various aspects of the objectives for designing a solution for the capstone project.	(i) Problem Analysis [K1, K2, K3, K4] Analyzed detection challenges like occlusion and lighting. Used theoretical knowledge to design SSL models. Evaluated models using precision, recall, and mAP. For Robotic Arm automatic	(i) Problem Analysis [EP1, EP2, EP3, EP6, EP7] Applied ML and robotics concepts. Faced issues in real-time detection, dataset limitations, and system performance. Stakeholders demand

		sorting, we have used other evaluation metrics too.	faster and more reliable automated systems. Limited access to technology and equipment presents implementation barriers.
CO4	Design and develop solutions for the capstone project that meets public health and safety, cultural, societal, and environmental considerations.	(i) Design and Implementation [K5] Designed the automated vegetable sorting framework using robotic arm and SSL models. Implemented YOLO-based system, applied DINOv2. Integrated Eigen-CAM visualizations to enhance interpretability. Created an application for real-time automated vegetable sorting, ensuring applicability in the food industry. Ensured proper training, testing, and hyperparameter tuning using a large custom dataset.	(i) Design and Implementation [EP1, EP2, EP4, EP5, EP6, EP7] Modern tools and ML used in system building [EP1]. Challenges faced due to limited resources, overfitting, and real-world constraints [EP2]. Struggled with control robotic arm framework dynamically [EP4]. Model privacy and deployment limitations [EP5]. Design considerations reflected different stakeholders' feedback and needs [EP6]. Availability of food industry experts remains a challenge [EP7].

CO5	Identify and apply modern engineering and IT tools for the design and development of the capstone project.	Materials and Devices [K6] Used Python and libraries like TensorFlow, Keras, NumPy, OpenCV, and Matplotlib for development. Leveraged custom dataset (multiple classes vegetable) and applied Eigen-CAM techniques. Applied hardware acceleration (GPU support) for model training. Developed an automated framework combining robotic arm with SSL models. All tools were chosen based on scalability and food industrial deployment potential.	Materials and Devices [EP1, EP2, EP4, EP5] Advanced IT tools used to develop and test models and robotic arm[EP1]. Alternative tools considered to improve performance or stability [EP2]. Lack of national access to advanced IT and automated tools [EP4]. Legal and ethical restrictions on data/model distribution [EP5].
CO6	Assess and address societal, health, safety, legal, and cultural aspects related to the implementation of the capstone project considering the relevant	Impact of Societal, Health, and Safety Issues [K7] Improves agricultural efficiency and reduces manual labor. No negative environmental	Impact of Societal, Health, and Safety Issues [EP2, EP5, EP6] Possible legal issues regarding data usage [EP2]. Trust issues around

	professional and engineering practices and solutions.	impact. No known harmful societal or environmental impact. Intended to support food industrial professionals, not replace them. Ensured the permission of field owners while taking images for the dataset. Cultural awareness considered for varying levels of technological acceptance in different regions. The framework aims to save time and labor costs by automated vegetable sorting that is the help of food industry professionals.	AI-based automation systems [EP5]. Differing cultural expectations around AI in food and agriculture can influence adoption and system acceptance [EP6].
--	---	--	--

CSE400-C

CO	CO Descriptions	K	EP
----	-----------------	---	----

CO7	Assess and address the sustainability and impact of the capstone project in societal and environmental contexts.	Results and Discussion [K7] The proposed framework of Robotic arm and SSL and its application do not have any negative impact on the economy, environment, or culture. In contrast, it contributes positively by sorting vegetables automatically, reducing labor costs, and improving agricultural efficiency. The solution is sustainable as it is designed to improve with more usage and data, increasing its performance over time.	Results and Discussion [EP2, EP5, EP6] Latency concerns among similar algorithms affect decision-making in critical agricultural settings [EP2]. Concerns remain around AI's ethical boundaries in sorting vegetables in all conditions [EP5]. Stakeholders—including food industry professionals—hold varied views on model trust and reliability [EP6].
CO8	Apply professional and engineering ethical principles and practices for the implementation of the capstone project	Results and Discussion [K7] All the data used in this project were collected by us from vegetable markets for research. We took permission before collecting the data and informed them it would be used only for research. There were no issues.	
CO9	Work effectively as an individual and a team member for successful completion of the capstone project	A strong collaborative effort was maintained throughout the development. Each team member was assigned specific tasks including data preprocessing, model building, explainability integration,	

		performance evaluation, and robotic arm integration. Team coordination ensured successful completion of the project milestones.	
CO10	Write effective reports and design documentation, and make effective presentations of the outcome of the capstone project		Final Capstone Project Report [EA1, EA2, EA3, EA4, EA5] Custom data was collected from vegetable markets. Hardware/software tools including TensorFlow, Keras, Streamlit, Robotic Arm and Google Colab were used. [EA1] Stakeholder discussions helped design the user-friendly, explainable interface for food professionals. [EA2] CSE knowledge in ML, DL, Robotic Arm and UI design was integrated into the development. EA4: The framework supports societal well-being through automated sorting of vegetables. [EA3]

			All legal, ethical, and privacy protocols were upheld throughout the project lifecycle. [EA5]
CO11	Conduct economic analysis and cost estimation; and apply appropriate project management processes in the development life cycle of the capstone project	The entire system, including SSL model and its deployment via dashboard, was developed using free, open-source tools to reduce cost. While robotic arm may be costly in the real-world, this project minimized costs by providing better framework solutions than other currently available literatures. Fieldwork, documentation, and team efforts were managed through a defined project plan with time allocation, version control, and regular syncs.	

CO12	Prepare to take part in independent and life-long learning for adapting emerging technologies for the solution of the complex computer science and engineering problems	Gained practical skills in AI, computer vision, and robotics, along with improved problem-solving ability and real-world system development experience.	
------	--	---	--